combinatorial
bell-numbers . . . 1
bell-numbers . . . 1
binomial-inversion . . . 1
burnside-lemma . . . 1
catalan . . . 2
derangements . . . 2
lucas . . . 2
pascal-triangle . . . 2
permutation-and-combination . . . 2
stars-and-bars . . . 2
stirling-numbers . . . 2
stirling-numbers . . . 3
vandermonde-identity . . . 3

dynamic-programming
bounded-subset-sum . . . 3
divide-and-conquer . . . 3
knuth . . . 3

extra
date-and-time . . . 3
divide-and-conquer . . . 4
macros . . . 4
masks . . . 4
permutation-to-int . . . 4
pragmas . . . 4
random . . . 4
ternary-search . . . 4

geometry
circle . . . 4
li-chao-tree . . . 4
line-container . . . 5
line . . . 5
manhattan-to-chebyshev . . . 5
point . . . 5
polar-sort . . . 5
polygon . . . 5
ray . . . 6
segment . . . 6

graph
binary-lifting . . . 6
centroid-decomposition . . . 6
articulations . . . 6
block-cut-tree . . . 7
bridge-tree . . . 7
bridges . . . 7
cycle . . . 7
dominator-tree . . . 7
steiner-tree . . . 7
tarjan . . . 7
two-sat . . . 8
bellman-ford . . . 8
floyd-warshall . . . 8
johnson . . . 8
potential-method . . . 8
dinic . . . 8
flow-theorems . . . 9
min-cost-k-flow . . . 9
hierholzer . . . 9
hld . . . 9
leaf-peeling . . . 10
link-cut-tree . . . 10
blossom . . . 10
hopcroft-karp . . . 11
hungarian . . . 11
kuhn . . . 11
matching-theorems . . . 11
matroid-intersection . . . 11
max-clique . . . 12
virtual-tree . . . 12

math
berlekamp-massey . . . 12
fast-fourier-transform . . . 12
fast-walsh-hadamard-transform . . . 13
modulo-transform-technique . . . 13
number-theoretic-transform . . . 13
gaussian-elimination . . . 13
xor-gaussian-elimination . . . 13
matrix . . . 13
nim . . . 13
xor-basis . . . 13

number-theory
floor-ceil . . . 14
floor-sum . . . 14
harmonic-lemma . . . 14
mobius-inversion . . . 14
mobius . . . 14
chinese-remainder-theorem . . . 15
exponent-reduction . . . 15
extended-gcd . . . 15
mod-log . . . 15
multi-extended-gcd . . . 15
linear-sieve . . . 15
miller-rabin . . . 15
pollard-rho . . . 15
segmented-sieve . . . 15
totient . . . 15

square-root
mo-3d . . . 15
mo-tree . . . 16
mo . . . 16
sqrt-decomposition . . . 16

string
aho-corasick . . . 16
dynamic-hasher . . . 16
hasher . . . 16
longest-prefix-suffix . . . 17
palindromic-tree . . . 17
prefix-automaton . . . 17
suffix-array . . . 17
tree-suffix-array . . . 17
suffix-automaton . . . 18
trie . . . 18
z-array . . . 18

structs
bit-2d . . . 18
bit . . . 18
range-bit . . . 18
sparse-bit-2d . . . 18
cover-tree . . . 18
k-cover-tree . . . 19
coprime-set . . . 19
interval-set . . . 19
range-set . . . 20
venice-set . . . 20
dsu-2d . . . 20
dsu . . . 20
dynamic-connectivity . . . 20
rollback-dsu . . . 20
dynamic-merge-sort-tree . . . 20
merge-sort-tree . . . 21
ordered-set . . . 21
iterative-segment-tree-2d . . . 21
iterative-segment-tree . . . 21
segment-tree-beats . . . 21
segment-tree . . . 21
sparse-segment-tree . . . 22
sparse-table-2d . . . 22
sparse-table . . . 22
treap . . . 22
wavelet-matrix . . . 23

combinatorial

bell-numbers

```cpp
vector<int> bell_numbers(int n) {
  vector<int> b = {1}, row = {1};
  for (int i = 1; i <= n; ++i) {
    vector<int> next = {row.back()};
    for (int x : row) next.push_back((next.back() + x) % mod);
    b.push_back(next[0]);
    row = next;
  }
  return b;
}
```

bell-numbers

Números de Bell

Definição. B_n é o número de formas de dividir n elementos distintos em grupos não vazios, sem ordem entre os grupos. Ex.: $B_3 = 5$: {123}, {12}{3}, {13}{2}, {23}{1}, {1}{2}{3}. Também conta n bolas distintas em caixas iguais (quantas caixas quiser), relações de equivalência em n elementos e formas de colorir n objetos quando só importa quem tem a mesma cor.

Valores. $B_0, \ldots, B_{10} = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$.

Como calcular

- **Recorrência:** o grupo do elemento $n+1$ leva junto alguns dos outros n, e os k que sobram se dividem de B_k formas:

$$\boxed{B_{n+1} = \sum_{k=0}^{n} \binom{n}{k} B_k}, \qquad B_0 = 1.$$

- **Triângulo de Bell** (sem binomial): a linha 0 é 1; cada linha começa com o último número da anterior, e cada número seguinte é o vizinho da esquerda mais o número acima desse vizinho. B_n é o primeiro da linha n, em $O(n^2)$:

1				
1	2			
2	3	5		
5	7	10	15	
15	20	27	37	52

binomial-inversion

```cpp
// Binomial Inversion: transforms subset group counts into exact
  frequency counts
// Time: O(n^2) | Space: O(n)
// Formula: exact[i] = sum_{j=i}^{n} (-1)^(j-i) * comb(j, i) *
  subsets[j]
vector<int> binomial_inversion(const vector<int> &subsets) {
  int n = subsets.size() - 1;
  vector<int> exact(n + 1);
  for (int i = 0; i <= n; ++i) {
    int cnt = 0;
    for (int j = i; j <= n; ++j) {
      int term = (comb(j, i) * subsets[j]) % mod;
      if ((j - i) & 1) {
        cnt = (cnt - term + mod) % mod;
      } else {
        cnt = (cnt + term) % mod;
      }
    }
    exact[i] = cnt;
  }
  return exact;
}
```

burnside-lemma

Lema de Burnside

Problema base. Contar quantas configurações são **diferentes** quando algumas são consideradas iguais por simetria. Ex.: colares que viram o mesmo colar quando giramos.

Ideia. Para cada simetria, conte quantas configurações continuam **iguais** depois de aplicá-la. A resposta é a média dessas contagens (cada configuração diferente é

contada exatamente $|G|$ vezes na soma):

$$\boxed{\text{resposta} = \frac{1}{|G|}\sum_{g\in G}\text{fixas}(g)}$$

- G: o conjunto de simetrias, incluindo "não fazer nada" (ex.: girar $0, 1, \ldots, n-1$ posições); $|G|$ é quantas são.
- g: uma simetria de G.
- fixas(g): quantas configurações não mudam quando aplicamos g.

Como contar fixas(g) **em colorações.** Siga para onde g leva cada posição: as posições se dividem em ciclos. Para a coloração não mudar, cada ciclo precisa ter uma cor só. Com $c(g)$ ciclos e k cores:

$$\text{fixas}(g) = k^{c(g)}.$$

Exemplo. Colares de 4 contas com 2 cores ($|G| = 4$ rotações):

- girar 0: cada posição é um ciclo, 4 ciclos: $2^4 = 16$;
- girar 1: $0 \to 1 \to 2 \to 3 \to 0$, 1 ciclo: $2^1 = 2$;
- girar 2: $(0\,2)(1\,3)$, 2 ciclos: $2^2 = 4$;
- girar 3: 1 ciclo: $2^1 = 2$.

Resposta: $(16 + 2 + 4 + 2)/4 = 6$, que são $0000, 0001, 0011, 0101, 0111, 1111$.

Casos clássicos (n posições, k cores)

- **Colar** (só girar, $|G| = n$): girar i posições forma $\gcd(i, n)$ ciclos. Ex.: $n = 6$, girar 2: $(0\,2\,4)(1\,3\,5)$, e $\gcd(2, 6) = 2$.

$$\frac{1}{n}\sum_{i=0}^{n-1} k^{\gcd(i,n)} = \frac{1}{n}\sum_{d\mid n}\varphi\left(\tfrac{n}{d}\right)k^d.$$

  A segunda forma agrupa as rotações pelo número de ciclos d: exatamente $\varphi(n/d)$ valores de i têm $\gcd(i, n) = d$. Ela só passa pelos divisores de n, útil para n grande.
- **Pulseira** (pode girar e virar, $|G| = 2n$): são as n rotações do colar mais n reflexões. Uma reflexão espelha num eixo: as contas em cima do eixo ficam paradas (ciclos de 1) e as outras trocam em pares (ciclos de 2). Com n ímpar, cada eixo passa por 1 conta: $\frac{n+1}{2}$ ciclos. Com n par, metade dos eixos passa por 2 contas ($\frac{n}{2} + 1$ ciclos) e metade passa entre contas ($\frac{n}{2}$ ciclos). Somando as reflexões em R:

$$\frac{1}{2n}\left(\sum_{i=0}^{n-1} k^{\gcd(i,n)} + R\right), \qquad R = \begin{cases} n\,k^{(n+1)/2}, & n \text{ ímpar} \\ \frac{n}{2}\left(k^{n/2+1} + k^{n/2}\right), & n \text{ par} \end{cases}$$

- **Grade** $n \times n$ (girar $0°, 90°, 180°, 270°$, $|G| = 4$): em $0°$ cada célula é um ciclo (n^2); em $90°$ e $270°$ as células andam em grupos de 4 ($\lceil n^2/4 \rceil$ ciclos); em $180°$ trocam em pares ($\lceil n^2/2 \rceil$ ciclos). O $\lceil\cdot\rceil$ conta o centro, que fica parado quando n é ímpar.

$$\frac{1}{4}\left(k^{n^2} + 2\,k^{\lceil n^2/4\rceil} + k^{\lceil n^2/2\rceil}\right).$$

catalan

```cpp
// Catalan (Ballot Theorem): number of grid paths from (0,0) to (l,r)
  staying x >= y at all times
// Time: O(1) | Space: O(1)
// Note: requires comb(n, k) | also counts valid parenthesis prefixes
  (l open, r close)
int catalan(int l, int r) {
  if ((l < 0 or r < 0) or l < r) return 0;
  return (comb(l + r, l) - comb(l + r, l + 1) + mod) % mod;
}
```

derangements

```cpp
// Derangements: number of permutations of N elements where no
  element stays in its place
// Time: O(N) | Space: O(N)
vector<int> derangements(int n) {
  vector<int> d(n + 1);
  d[0] = 1;
  for (int i = 2; i <= n; ++i) d[i] = (i - 1) * (d[i - 1] + d[i - 2])
    % mod;
  return d;
}
```

lucas

```cpp
int lucas(int n, int k) {
  if (k == 0 or k == n) return 1;
  if (k > n) return 0;
  return (lucas(n / mod, k / mod) * comb(n % mod, k % mod)) % mod;
}
```

pascal-triangle

Triângulo de Pascal

Linha n, coluna k: $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1															
1	1	1														
2	1	2	1													
3	1	3	3	1												
4	1	4	6	4	1											
5	1	5	10	10	5	1										
6	1	6	15	20	15	6	1									
7	1	7	21	35	35	21	7	1								
8	1	8	28	56	70	56	28	8	1							
9	1	9	36	84	126	126	84	36	9	1						
10	1	10	45	120	210	252	210	120	45	10	1					
11	1	11	55	165	330	462	462	330	165	55	11	1				
12	1	12	66	220	495	792	924	792	495	220	66	12	1			
13	1	13	78	286	715	1287	1716	1716	1287	715	286	78	13	1		
14	1	14	91	364	1001	2002	3003	3432	3003	2002	1001	364	91	14	1	
15	1	15	105	455	1365	3003	5005	6435	6435	5005	3003	1365	455	105	15	1

permutation-and-combination

Permutação e Combinação

$$\boxed{P(n,k) = \frac{n!}{(n-k)!}} \qquad \boxed{\binom{n}{k} = \frac{n!}{k!\,(n-k)!}}$$

stars-and-bars

Stars and Bars

Problema base. Quantas soluções inteiras tem

$$x_1 + x_2 + \cdots + x_k = n, \qquad x_i \geq 0\ ?$$

Ideia. Represente n como n estrelas ($\star$) e separe-as em k grupos usando $k-1$ barras ($|$). Cada arrumação de n estrelas e $k-1$ barras em fila corresponde a exatamente uma solução (o grupo i é a quantidade de estrelas entre a barra $i-1$ e a barra i). Ex.: $n = 5$, $k = 3$: $\star\star\,|\,\star\,|\,\star\star \leftrightarrow (2, 1, 2)$ e $||\star\star\star\star\star \leftrightarrow (0, 0, 5)$. Basta escolher as posições das barras entre $n + k - 1$ posições:

$$\boxed{\binom{n+k-1}{k-1}}$$

Variações

- **Variáveis positivas** ($x_i \geq 1$): coloque as barras nos $n-1$ espaços entre estrelas, no máximo uma por espaço:

$$\binom{n-1}{k-1}.$$

- **Limite inferior** ($x_i \geq l_i$): substitua $y_i = x_i - l_i \geq 0$. Recai no caso base com $n' = n - \sum l_i$ (se $n' < 0$, a resposta é 0):

$$\binom{n'+k-1}{k-1}.$$

- **Soma com desigualdade** ($x_1 + \cdots + x_k \leq n$, $x_i \geq 0$): adicione uma variável de folga $x_{k+1} = n - \sum x_i \geq 0$. Agora são $k+1$ variáveis com soma exatamente n:

$$\binom{n+k}{k}.$$

- **Limite superior** ($0 \leq x_i \leq u_i$): inclusão-exclusão. Para um conjunto S de variáveis que *violam* o limite, force $x_i \geq u_i + 1$ para $i \in S$:

$$\sum_{S\subseteq\{1,\ldots,k\}} (-1)^{|S|}\binom{n - \sum_{i\in S}(u_i+1) + k - 1}{k-1},$$

  considerando o termo 0 quando o topo fica negativo.
- **Mesmo limite superior** ($0 \leq x_i \leq c$ para todo i): os conjuntos S de mesmo tamanho contribuem igual, então

$$\sum_{j=0}^{k}(-1)^j\binom{k}{j}\binom{n - j(c+1) + k - 1}{k-1},$$

  parando quando $n - j(c+1) < 0$.

Problemas equivalentes

A mesma fórmula $\binom{n+k-1}{k-1} = \binom{n+k-1}{n}$ conta:

- distribuir n objetos **idênticos** em k caixas **distintas**;
- **multiconjuntos** de tamanho n com elementos de k tipos (combinação com repetição);
- sequências **não-decrescentes** de tamanho n com valores em $[1, k]$ (x_i = quantas vezes o valor i aparece);
- monômios de grau n em k variáveis.

stirling-numbers

```cpp
// Stirling Numbers of the First Kind: t[i][k] = permutations of i
  elements with exactly k cycles, for i, k <= n
// Time: O(N^2) | Space: O(N^2)
// Note: uses mod (any) | c(i, k) = (i - 1) c(i - 1, k) + c(i - 1, k
  - 1) | signed = (-1)^(i - k) t[i][k]
vector<vector<int>> stirling1_table(int n) {
  vector<vector<int>> t(n + 1, vector<int>(n + 1));
  t[0][0] = 1;
  for (int i = 1; i <= n; ++i) {
    for (int k = 1; k <= i; ++k) t[i][k] = (t[i - 1][k - 1] + (i - 1)
      * t[i - 1][k]) % mod;
  }
  return t;
}

// Stirling Numbers of the Second Kind: t[i][k] = ways to split i
  elements into exactly k non-empty groups, for i, k <= n
// Time: O(N^2) | Space: O(N^2)
// Note: uses mod (any) | S(i, k) = k S(i - 1, k) + S(i - 1, k - 1)
vector<vector<int>> stirling2_table(int n) {
  vector<vector<int>> t(n + 1, vector<int>(n + 1));
  t[0][0] = 1;
  for (int i = 1; i <= n; ++i) {
    for (int k = 1; k <= i; ++k) t[i][k] = (t[i - 1][k - 1] + k * t[i
      - 1][k]) % mod;
  }
  return t;
}

// Stirling Row of the First Kind: row[k] = c(n, k) for k <= n, the
  coefficients of x (x + 1) ... (x + n - 1)
// Time: O(N log^2 N) | Space: O(N)
// Note: uses mod and convolution (mod must work with the NTT) |
  multiplies the factors (x + i) for i in [l, r) by divide and conquer
vector<int> stirling1_row(int n, int l = 0, int r = -1) {
  if (r == -1) r = n;
  if (l == r) return {1};
  if (r - l == 1) return {l, 1};
  int m = (l + r) / 2;
  return convolution<mod>(stirling1_row(n, l, m), stirling1_row(n, m,
    r));
}

// Stirling Row of the Second Kind: row[k] = S(n, k) for k <= n, with
  one convolution
// Time: O(N log N) | Space: O(N)
// Note: uses mod, mod_pow, invf (inverse factorials up to n) and
  convolution (mod must work with the NTT) | convolves (-1)^i / i!
  with j^n / j!
vector<int> stirling2_row(int n) {
  vector<int> a(n + 1), b(n + 1);
  for (int i = 0; i <= n; ++i) {
    a[i] = i % 2 == 0 ? invf[i] : mod - invf[i];
    b[i] = mod_pow(i, n, mod) * invf[i] % mod;
  }
  vector<int> row = convolution<mod>(a, b);
  row.resize(n + 1);
  return row;
}
```

```cpp
// Stirling Number of the Second Kind: S(n, k) for a single pair, by
  inclusion-exclusion on the empty groups
// Time: O(k log n) | Space: O(1)
// Note: uses mod (prime), mod_pow and invf (inverse factorials up to
  k) | S(n, k) = sum of (-1)^(k - j) j^n / (j! (k - j)!)
int stirling2(int n, int k) {
  int s = 0;
  for (int j = 0; j <= k; ++j) {
    int term = mod_pow(j, n, mod) * invf[j] % mod * invf[k - j] % mod;
    s = (k - j) % 2 == 0 ? (s + term) % mod : (s - term + mod) % mod;
  }
  return s;
}
```

stirling-numbers

Números de Stirling

Notação. $c(n,k)$ (primeiro tipo, sem sinal) é o número de permutações de n elementos com exatamente k ciclos. $S(n,k)$ (segundo tipo) é o número de formas de dividir n elementos distintos em exatamente k grupos não vazios, sem ordem entre os grupos. $x^{\underline{k}} = x(x-1)\cdots(x-k+1)$ e $x^{\overline{k}} = x(x+1)\cdots(x+k-1)$.

Segundo tipo

- **Recorrência:** o elemento n entra num dos k grupos ou forma um grupo sozinho: $S(n,k) = k\,S(n-1,k) + S(n-1,k-1)$, com $S(0,0)=1$.
- **Fórmula:** inclusão-exclusão nas caixas vazias, dividida por $k!$ porque os grupos não têm ordem:

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

  Um valor sai em $O(k \log n)$. A linha inteira é a convolução de $\frac{(-1)^i}{i!}$ com $\frac{j^n}{j!}$.
- **Usos:** $k!\,S(n,k)$ conta as funções sobrejetoras de n em k elementos (bolas distintas em caixas distintas, nenhuma vazia); $\sum_k S(n,k) = B_n$ (Bell); $x^n = \sum_k S(n,k)\,x^{\underline{k}}$ troca potências por fatoriais caindo.

Primeiro tipo

- **Recorrência:** o elemento n forma um ciclo sozinho ou entra logo depois de um dos outros $n-1$: $c(n,k) = (n-1)\,c(n-1,k) + c(n-1,k-1)$, com $c(0,0)=1$.
- **Polinômio:**

$$x^{\overline{n}} = x(x+1)\cdots(x+n-1) = \sum_k c(n,k)\,x^k$$

  A linha inteira sai multiplicando os n fatores por divisão e conquista com NTT, em $O(n \log^2 n)$.
- **Com sinal:** $s(n,k) = (-1)^{n-k}c(n,k)$ são os coeficientes de $x^{\underline{n}} = \sum_k s(n,k)\,x^k$.
- $\sum_k c(n,k) = n!$.

Valores especiais

$c(n,1) = (n-1)!$, $S(n,2) = 2^{n-1}-1$, $c(n,n-1) = S(n,n-1) = \binom{n}{2}$ e $c(n,n) = S(n,n) = 1$.

vandermonde-identity

Identidade de Vandermonde

Problema base. Trocar uma soma de produtos de binomiais, $O(N)$, por um único binomial, $O(1)$ com fatoriais pré-computados.
Ideia. Escolher r pessoas de um grupo com m homens e n mulheres é somar, para cada k, as formas de escolher k homens e $r-k$ mulheres:

$$\sum_k \binom{m}{k}\binom{n}{r-k} = \binom{m+n}{r}$$

A soma é sobre todo k (termos fora do intervalo valem 0). Para aplicar, **os índices de baixo precisam somar uma constante** $(k + (r-k) = r)$. Quando não somam, use a simetria $\binom{n}{k} = \binom{n}{n-k}$ em um dos termos.

Padrões

- **Índices iguais:** em $\sum_k \binom{x}{k}\binom{y}{k}$ os de baixo somam $2k$, que não é constante.

  Trocando $\binom{x}{k}$ por $\binom{x}{x-k}$, eles somam $(x-k)+k = x$:

$$\sum_k \binom{x}{k}\binom{y}{k} = \sum_k \binom{x}{x-k}\binom{y}{k} = \binom{x+y}{x}.$$

  Caso particular: $\sum_k \binom{n}{k}^2 = \binom{2n}{n}$.
- **Índices deslocados:** em $\sum_k \binom{x}{k+a}\binom{y}{k+b}$, troque $\binom{x}{k+a}$ por $\binom{x}{x-k-a}$; os de baixo somam $(x-k-a)+(k+b) = x-a+b$:

$$\sum_k \binom{x}{k+a}\binom{y}{k+b} = \binom{x+y}{x-a+b}.$$

  Ex.: $\sum_k \binom{x}{k-1}\binom{y}{k} = \binom{x+y}{x+1}$ $(a=-1,\ b=0)$.
- **Índices de cima com soma constante:** escolha $a+b+1$ números de $\{0,\dots,n\}$ e olhe a posição k do $(a+1)$-ésimo:

$$\sum_{k=0}^{n}\binom{k}{a}\binom{n-k}{b} = \binom{n+1}{a+b+1}.$$

- **Hockey stick** (o caso $b=0$ acima):

$$\sum_{i=0}^{n}\binom{i}{k} = \binom{n+1}{k+1}.$$

Implementação: pré-compute fatoriais e inversos modulares para calcular $\binom{n}{k} \bmod p$ em $O(1)$.

dynamic-programming

bounded-subset-sum

```cpp
// Bounded Subset Sum: maximizes subset sum without exceeding the
  target
// Time: O(N * max(W)) | Space: O(max(W))
// Note: efficient for huge targets with small weights
int bounded_subset_sum(const vector<int> &w, int target) {
  int n = w.size(), k = 0, s = 0;
  while (k < n and s + w[k] <= target) {
    s += w[k++];
  }
  if (k == n) return s;
  int m = *max_element(w.begin(), w.end());
  vector<int> pre;
  vector<int> cur(2 * m, -1);
  cur[s - target + m] = k;
  for (int i = k; i < n; ++i) {
    pre = cur;
    for (int x = 0; x < m; ++x) {
      cur[x + w[i]] = max(cur[x + w[i]], pre[x]);
    }
    for (int x = 2 * m - 1; x > m; --x) {
      for (int j = max<int>(0, pre[x]); j < cur[x]; ++j) {
        cur[x - w[j]] = max(cur[x - w[j]], j);
      }
    }
  }
  int x = m;
  while (cur[x] == -1) x--;
  return x + target - m;
}
```

divide-and-conquer

```cpp
// Divide and Conquer: minimum cost to split 1..n into k contiguous
  groups, where group [l, r] costs cost(l, r)
// Time: O(K N log N) calls of cost | Space: O(N)
// Note: needs cost(l, r) and inf | works when the best end of the
  first group never moves left as its start grows (quadrangle
  inequality)
int divide_and_conquer(int n, int k) {
  vector<int> prev(n + 2, inf), curr(n + 2, inf);
  prev[n + 1] = 0;
  auto solve = [&](auto &self, int l, int r, int low, int high) ->
    void {
    if (l > r) return;
    int m = (l + r) / 2, opt = high;
    curr[m] = inf;
    for (int i = max(m, low); i <= high; ++i) if (prev[i + 1] < inf) {
      int val = cost(m, i) + prev[i + 1];
      if (val < curr[m]) curr[m] = val, opt = i;
    }
    self(self, l, m - 1, low, opt);
    self(self, m + 1, r, opt, high);
  };
  for (int i = 1; i <= k; ++i) {
    solve(solve, 1, n, 1, n);
    curr[n + 1] = inf;
    swap(prev, curr);
  }
  return prev[1];
}
```

knuth

```cpp
int knuth(int n) {
  vector<vector<int>> ans(n + 2, vector<int>(n + 2));
  vector<vector<int>> opt(n + 2, vector<int>(n + 2));
  for (int l = n; l >= 1; --l) {
    ans[l][l] = cost(l, l);
    opt[l][l] = l;
    for (int r = l + 1; r <= n; ++r) {
      int c = cost(l, r);
      ans[l][r] = inf;
      for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l + 1][r]);
        ++k) {
        int val = ans[l][k] + ans[k + 1][r] + c;
        if (val < ans[l][r]) ans[l][r] = val, opt[l][r] = k;
      }
    }
  }
  return ans[1][n];
}
```

extra

date-and-time

```cpp
// Date and Time: dates as day, month, year and clock times as seconds
// Time: O(1) per operation | Space: O(1)
string pad(int x, int w) {
  string s = to_string(x);
  while (s.size() < w) s = "0" + s;
  return s;
}

// "hh:mm:ss" -> seconds since 00:00:00
int to_seconds(const string &t) {
  int h, m, s;
  char c;
  stringstream(t) >> h >> c >> m >> c >> s;
  return h * 3600 + m * 60 + s;
}

// seconds -> "hh:mm:ss" (to_clock((x % 86400 + 86400) % 86400) wraps
  around midnight)
string to_clock(int x) {
  return pad(x / 3600, 2) + ":" + pad(x / 60 % 60, 2) + ":" + pad(x %
    60, 2);
}

// "dd/mm/yyyy" -> [d, m, y]
array<int, 3> parse(const string &t) {
  int d, m, y;
  char c;
  stringstream(t) >> d >> c >> m >> c >> y;
  return {d, m, y};
}

// [d, m, y] -> "dd/mm/yyyy"
string show(int d, int m, int y) {
  return pad(d, 2) + "/" + pad(m, 2) + "/" + pad(y, 4);
}

bool is_leap(int y) {
  return (y % 4 == 0 and y % 100 != 0) or y % 400 == 0;
}

int month_days(int m, int y) {
  const int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
  return days[m - 1] + (m == 2 and is_leap(y));
}

// number of the day d/m/y, counting from 01/01/0001 = 1
int to_days(int d, int m, int y) {
  const int before[] = {0, 31, 59, 90, 120, 151, 181, 212, 243, 273,
    304, 334};
  int p = y - 1;
  return p * 365 + p / 4 - p / 100 + p / 400 + before[m - 1] + (m > 2
    and is_leap(y)) + d;
}

// [d, m, y] of the day number z, inverse of to_days
array<int, 3> from_days(int z) {
  int y = z * 400 / 146097 + 1;
  while (to_days(1, 1, y) > z) y--;
  while (to_days(1, 1, y + 1) <= z) y++;
  z -= to_days(1, 1, y) - 1;
  int m = 1;
  while (z > month_days(m, y)) z -= month_days(m++, y);
  return {z, m, y};
```

```
}

int weekday(int d, int m, int y) {
  return (to_days(d, m, y) - 1) % 7;
}

array<int, 3> add_days(int d, int m, int y, int k) {
  return from_days(to_days(d, m, y) + k);
}

int days_between(int d1, int m1, int y1, int d2, int m2, int y2) {
  return to_days(d2, m2, y2) - to_days(d1, m1, y1);
}

array<int, 3> add_months(int d, int m, int y, int k) {
  int t = y * 12 + m - 1 + k;
  y = t / 12, m = t % 12 + 1;
  return {min(d, month_days(m, y)), m, y};
}
```

divide-and-conquer

```
// Divide and Conquer: answers offline range queries [i, j] at the
  middle m of the first range that splits them
// Time: O((N + Q) log N) | Space: O(N + Q)
// Note: suf[i] = answer of arr[i..m] and pre[j] = answer of
  arr[m+1..j] | edit both sweeps and suf[i] + pre[j]
vector<int> divide_and_conquer(vector<int> &arr, vector<array<int,
  2>> &queries) {
  int n = arr.size();
  vector<int> suf(n + 1), pre(n + 1), ans(queries.size());
  vector<array<int, 3>> q;
  for (int k = 0; k < queries.size(); ++k)
    q.push_back({queries[k][0], queries[k][1], k});
  auto rec = [&](auto &self, int l, int r, vector<array<int, 3>> &q)
    -> void {
    if (l > r or q.empty()) return;
    int m = (l + r) / 2;
    suf[m + 1] = 0;
    for (int i = m; i >= l; --i) suf[i] = arr[i] + suf[i + 1];
    pre[m] = 0;
    for (int j = m + 1; j <= r; ++j) pre[j] = pre[j - 1] + arr[j];
    vector<array<int, 3>> ql, qr;
    for (auto [i, j, k] : q) {
      if (j < m) ql.push_back({i, j, k});
      else if (i > m) qr.push_back({i, j, k});
      else ans[k] = suf[i] + pre[j];
    }
    q.clear(), q.shrink_to_fit();
    self(self, l, m - 1, ql);
    self(self, m + 1, r, qr);
  };
  rec(rec, 0, n - 1, q);
  return ans;
}
```

macros

```
void scan(auto&... args) {(cin >> ... >> args);}
void print(auto&&... args) {(cout << ... << args);}
```

masks

```
void submasks(int mask) {
  for (int sub = mask; sub > 0; sub = mask & (sub - 1)) {
    process(sub);
  }
  process(0);
}

vector<int> sum_over_subsets(int n, vector<int> f) {
  for (int i = 0; i < n; ++i) {
    int bit = 1 << i;
    for (int mask = 0; mask < (1 << n); ++mask) if (mask & bit) {
      f[mask] += f[mask ^ bit];
    }
  }
  return f;
}

vector<int> sum_over_supersets(int n, vector<int> f) {
  for (int i = 0; i < n; ++i) {
    int bit = 1 << i;
    for (int mask = 0; mask < (1 << n); ++mask) if (not (mask & bit))
      {
      f[mask] += f[mask | bit];
    }
  }
  return f;
}
```

permutation-to-int

```
int permutation_to_int(vector<int> &p) {
  int mask = 0;
  int i = 0, r = 0;
  for (int x : p) {
    r = r * ++i + __builtin_popcountll(mask & -(1LL << x));
    mask |= 1LL << x;
  }
  return r;
}
```

pragmas

```
#pragma GCC optimize("O3", "unroll-loops")
#pragma GCC target("avx2", "bmi", "bmi2", "lzcnt", "popcnt")
```

random

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
```

ternary-search

```
int ternary_search(int l, int r, bool is_max) {
  int a = l, b = r;
  while (l < r) {
    int m = l + (r - l) / 2;
    bool is_up = fun(m) < fun(m + 1);
    if (is_up == is_max) l = m + 1;
    else r = m;
  }
  for (int x : {a, b}) {
    bool is_up = fun(l) < fun(x);
    if (is_up == is_max) l = x;
  }
  return l;
}

float ternary_search(float l, float r, bool is_max) {
  for (int i = 0; i < 100; ++i) {
    float m1 = l + (r - l) / 3;
    float m2 = r - (r - l) / 3;
    bool is_up = fun(m1) < fun(m2);
    if (is_up == is_max) l = m1;
    else r = m2;
  }
  return (l + r) / 2;
}
```

geometry

circle

```
point incircle(point a, point b, point c) {
  T ab = dist(a, b);
  T bc = dist(b, c);
  T ac = dist(a, c);
  T p = ab + bc + ac;
  if (p == 0) return {};
  return (a * bc + b * ac + c * ab) / p;
}

point circumcircle(point a, point b, point c) {
  point v = b - a;
  point w = c - a;
  T h = cross(v, w);
  if (h == 0) return {};
  return a + perp(v * norm(w) - w * norm(v)) / (2.0 * h);
}

vector<point> circle_line_inter(point o, T r, line l) {
  T s = l.side(o);
  T d2 = (s * s) / norm(l.v);
  T h2 = r * r - d2;
  if (h2 < 0) return {};
  point p = l.proj(o);
  if (h2 == 0) return {p};
  point h = l.v * sqrt(h2 / norm(l.v));
  return {p - h, p + h};
}

vector<point> circle_segment_inter(point o, T r, point a, point b) {
  if (a == b) {
    if (norm(a - o) == r * r) return {a};
    return {};
  }
  vector<point> l = circle_line_inter(o, r, line(a, b));
  vector<point> h;
  for (point p : l) if (in_disk(a, b, p)) h.push_back(p);
  return h;
}

T circle_inter_area(point o1, T r1, point o2, T r2) {
  T d = dist(o1, o2);
  if (d >= r1 + r2) return 0.0;
  if (d <= abs(r1 - r2)) return pi * min(r1, r2) * min(r1, r2);
  T h1 = 2.0 * acos((d * d + r1 * r1 - r2 * r2) / (2.0 * d * r1));
  T h2 = 2.0 * acos((d * d + r2 * r2 - r1 * r1) / (2.0 * d * r2));
  T a1 = 0.5 * r1 * r1 * (h1 - sin(h1));
  T a2 = 0.5 * r2 * r2 * (h2 - sin(h2));
  return a1 + a2;
}

vector<point> circle_inter(point o1, T r1, point o2, T r2) {
  point v = o2 - o1;
  T d2 = norm(v);
  if (d2 == 0) return {};
  T op = (d2 + r1 * r1 - r2 * r2) / 2.0;
  T h2 = r1 * r1 - (op * op) / d2;
  if (h2 < 0) return {};
  point p = o1 + v * (op / d2);
  if (h2 == 0) return {p};
  point h = perp(v) * sqrt(h2 / d2);
  return {p - h, p + h};
}
```

li-chao-tree

```
// Li Chao Tree: dynamic line & segment container for maximum convex
  hull trick
// Time: line insert/query O(log U) | segment insert O(log^2 U) |
  Space: O(N log U)
// Note: computes max(ax + b) | for min: insert (-a, -b) and take
  -query(x)
struct li_chao_tree {
private:
  struct line {
    int a, b;

    int operator()(int x) const {
      return a * x + b;
    }
  };

  struct node {
    line f;
    int l, r;
  };

  int min_x, max_x;
  vector<node> tree;

  int create_node() {
    tree.push_back({{0, -inf}, 0, 0});
    return tree.size() - 1;
  }

  int insert_line(line f, int p, int l, int r) {
    if (p == 0) p = create_node();
    int m = l + (r - l) / 2;
    bool is_l = f(l) > tree[p].f(l);
    bool is_m = f(m) > tree[p].f(m);
    if (is_m) swap(f, tree[p].f);
    if (l == r) return p;
    if (is_l != is_m) tree[p].l = insert_line(f, tree[p].l, l, m);
    else tree[p].r = insert_line(f, tree[p].r, m + 1, r);
    return p;
  }

  int insert_segment(line f, int ql, int qr, int p, int l, int r) {
    if (p == 0) p = create_node();
    if (ql <= l and r <= qr) return insert_line(f, p, l, r);
    int m = l + (r - l) / 2;
    if (ql <= m) tree[p].l = insert_segment(f, ql, qr, tree[p].l, l,
      m);
    if (qr > m) tree[p].r = insert_segment(f, ql, qr, tree[p].r, m +
      1, r);
    return p;
  }

  int query(int x, int p, int l, int r) {
    if (p == 0) return -inf;
    int y = tree[p].f(x);
    if (l == r) return y;
    int m = l + (r - l) / 2;
    if (x <= m) y = max(y, query(x, tree[p].l, l, m));
    else y = max(y, query(x, tree[p].r, m + 1, r));
    return y;
  }

public:
  li_chao_tree(int min_x, int max_x): min_x(min_x), max_x(max_x) {
    tree.push_back({{0, -inf}, 0, 0});
    create_node();
  }

  void insert_line(int a, int b) {
    insert_line({a, b}, 1, min_x, max_x);
  }

  void insert_segment(int a, int b, int l, int r) {
    insert_segment({a, b}, l, r, 1, min_x, max_x);
  }
```

```
  int query(int x) {
    return query(x, 1, min_x, max_x);
  }
};
```

line-container

```
// Line Container: dynamic convex hull trick for maximum
// Time: insert/query O(log N) | Space: O(N)
// Note: computes max(ax + b) | change '<' to '>' in line operator
  for min
struct line {
  mutable int a, b, last;

  bool operator<(const line &o) const {
    return tie(a, b) < tie(o.a, o.b);
  }

  bool operator<(int x) const {
    return last < x;
  }
};

struct line_container : multiset<line, less<>> {
  int floor(int n, int d) {
    return n / d - ((n ^ d) < 0 and n % d != 0);
  }

  bool intersect(iterator s, iterator t) {
    if (t == end()) return s->last = inf, false;
    if (s->a == t->a) s->last = -inf;
    else s->last = floor(t->b - s->b, s->a - t->a);
    return s->last >= t->last;
  }

  void insert(int a, int b) {
    auto curr = multiset::insert({a, b, 0}), after = next(curr),
      before = curr;
    while (intersect(curr, after)) after = erase(after);
    if (before != begin() and intersect(--before, curr))
      intersect(before, curr = erase(curr));
    while ((curr = before) != begin() and (--before)->last >=
      curr->last) intersect(before, erase(curr));
  }

  int query(int x) {
    auto l = lower_bound(x);
    return l->a * x + l->b;
  }
};
```

line

```
struct line {
  point v;
  T c;

  line(point v, T c): v(v), c(c) {}

  line(T a, T b, T c): v({b, -a}), c(c) {}

  line(point p, point q): v(q - p), c(cross(q, p)) {}

  T side(point p) {
    return cross(v, p) - c;
  }

  float dist(point p) {
    return abs(side(p)) / sqrt(norm(v));
  }

  bool comp(point p, point q) {
    return dot(v, p) < dot(v, q);
  }

  // Projection: calculates the orthogonal projection of point p onto
    the line
  // Note: translates p perpendicularly to the line (assumes perp is
    counter-clockwise)
  point proj(point p) {
    return p - (perp(v) * side(p)) / norm(v);
  }

  // Reflection: finds the mirror image of point p across the line
  // Note: projects p and extends it by the same distance
  point refl(point p) {
    return p - (2.0 * perp(v) * side(p)) / norm(v);
  }
};

vector<point> line_inter(line l, line r) {
  if (cross(l.v, r.v) == 0) return {};
  return {(l.c * r.v - r.c * l.v) / cross(l.v, r.v)};
}

vector<point> line_segment_inter(line l, point a, point b) {
  T sa = l.side(a);
  T sb = l.side(b);
  if (sign(sa) * sign(sb) > 0) return {};
  if (sa == 0 and sb == 0) {
    vector<point> p = {a, b};
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    return p;
  }
  return {(a * sb - b * sa) / (sb - sa)};
}
```

manhattan-to-chebyshev

Manhattan para Chebyshev

Problema base. A distância de Manhattan entre (x_1, y_1) e (x_2, y_2) é $|x_1 - x_2| + |y_1 - y_2|$. A soma junta as duas coordenadas, então não dá para tratar x e y separadamente (ex.: achar o par mais distante sem testar todos os pares).

Ideia. Gire o plano em 45° com $u = x + y$ e $v = x - y$. Como $|a| + |b| = \max(|a + b|, |a - b|)$, Manhattan vira Chebyshev, que é a maior das diferenças:

$$\boxed{|x_1 - x_2| + |y_1 - y_2| = \max(|u_1 - u_2|,\ |v_1 - v_2|)}$$

Com o max, cada eixo (u ou v) pode ser analisado sozinho.

Variações

- **Volta (Chebyshev para Manhattan):** com o mesmo u, v,

$$2\max(|x_1 - x_2|,\ |y_1 - y_2|) = |u_1 - u_2| + |v_1 - v_2|.$$

- **Paridade:** u e v têm sempre a mesma paridade; ao voltar com $x = \frac{u+v}{2}$, $y = \frac{u-v}{2}$, um (u, v) de paridades diferentes não vem de ponto inteiro.

Aplicações

- **Par mais distante** entre N pontos em $O(N)$: $\max(\max u - \min u,\ \max v - \min v)$.
- **Pontos a distância $\leq d$ de (x_0, y_0):** o losango vira o quadrado $|u - u_0| \leq d$, $|v - v_0| \leq d$ (com $u_0 = x_0 + y_0$, $v_0 = x_0 - y_0$), alinhado aos eixos (prefix sum 2D, BIT, sweep).
- **Soma das distâncias entre todos os pares:** Manhattan já separa por eixo; para Chebyshev, use a volta e divida por 2. Em cada eixo, ordene e some com prefix sums.

point

```
#define x real()
#define y imag()

using T = int;
using point = complex<T>;

namespace std {
  bool operator<(const point &a, const point &b) {
    return pair(a.x, a.y) < pair(b.x, b.y);
  }
}

istream &operator>>(istream &is, point &p) {
  T r, i; is >> r >> i;
  p = {r, i};
  return is;
}

int sign(T n) {
  if (n == 0) return 0;
  return n < 0 ? -1 : +1;
}

float dist(point a, point b) {
  return sqrt(norm(a - b));
}

T dot(point p, point q) {
  return (conj(p) * q).x;
}

T cross(point p, point q) {
  return (conj(p) * q).y;
}

T orient(point a, point b, point c) {
  return cross(b - a, c - a);
}

point perp(point p) {
  return {-p.y, p.x};
}

// Directed Angle: shortest angle from vector v to vector w
// Note: returns radians in [-pi, pi], negative result means w is to
  the right
float angle(point v, point w) {
  return atan2(cross(v, w), dot(v, w));
}

float closest_pair(vector<point> p) {
  int n = p.size();
  sort(p.begin(), p.end());
  auto compare = [](const point &a, const point &b) {
    if (a.y != b.y) return a.y < b.y;
    return a.x < b.x;
  };
  multiset<point, decltype(compare)> active(compare);
  active.insert(p[0]);
  float d = inf;
  int l = 0;
  for (int i = 1; i < n; ++i) {
    while (l < i and p[i].x - p[l].x >= d) {
      active.erase(active.find(p[l]));
      l++;
    }
    auto begin = active.lower_bound({p[i].x - d, p[i].y - d});
    auto end = active.upper_bound({p[i].x + d, p[i].y + d});
    for (auto itr = begin; itr != end; ++itr) {
      float c = dist(*itr, p[i]);
      d = min(d, c);
    }
    active.insert(p[i]);
  }
  return d;
}
```

polar-sort

```
// Is Upper: determines which half-plane a point lies in relative to
  ray v
// Note: returns 1 for (0, pi] (upper half and opposite ray), 0 for
  (pi, 2pi) (lower half, ray v and origin)
bool is_upper(point p, point v) {
  T c = cross(v, p);
  T d = dot(v, p);
  return c > 0 or (c == 0 and d < 0);
}

// Polar Sort: sorts points counter-clockwise starting from the
  opposite of reference ray v
// Time: O(N log N) | Space: O(1)
// Note: colinear points sorted by distance
void polar_sort(vector<point> &p, point o = {0, 0}, point v = {1, 0})
  {
  sort(p.begin(), p.end(), [&](point a, point b) {
    a = a - o, b = b - o;
    bool ha = is_upper(a, v), hb = is_upper(b, v);
    if (ha != hb) return ha < hb;
    T c = cross(a, b);
    if (c != 0) return c > 0;
    return norm(a) < norm(b);
  });
}
```

polygon

```
float area(point a, point b, point c) {
  return abs(cross(b - a, c - a)) / 2.0;
}

float area(const vector<point> &p) {
  T a = 0;
  int n = p.size();
  for (int i = 0; i < n; ++i) a += cross(p[i], p[(i + 1) % n]);
  return abs(a) / 2.0;
}

// Pick's Theorem: calculates the area of a simple polygon with
  integer vertices
// Time: O(1) | Space: O(1)
// Note: i is the number of interior points, b is the number of
  boundary points
float picks_theorem(int i, int b) {
  return i + (b / 2.0) - 1;
}

// Boundary Points: calculates the total number of integer points on
  the boundary of a polygon
```

```
// Time: O(N log M) | Space: O(1)
// Note: relies on the property that the number of integer points on
  a segment is gcd(dx, dy)
int boundary_points(const vector<point> &g) {
  int c = 0, n = g.size();
  for (int i = 0; i < n; ++i) {
    point p = g[i], q = g[(i + 1) % n];
    c += gcd(p.x - q.x, p.y - q.y);
  }
  return c;
}

// Winding Number: computes how many times a polygon wraps around a
  point
// Time: O(N) | Space: O(1)
// Note: returns non-zero for inside, zero for outside or boundary
int winding_number(const vector<point> &g, point a) {
  int w = 0, n = g.size();
  for (int i = 0; i < n; ++i) {
    point p = g[i], q = g[(i + 1) % n];
    if (on_segment(p, q, a)) return 0;
    int d = (q.y >= a.y) - (p.y >= a.y);
    if (d * orient(p, q, a) > 0) w += d;
  }
  return w;
}

vector<point> convex_hull(vector<point> g, bool col) {
  sort(g.begin(), g.end());
  g.erase(unique(g.begin(), g.end()), g.end());
  int n = g.size();
  if (n < 3) return g;
  bool all = col;
  for (int i = 2; i < n and all; ++i) {
    all &= orient(g[0], g[1], g[i]) == 0;
  }
  if (all) return g;
  vector<point> h;
  T lim = col ? 0 : 1;
  for (int i = 0; i <= 1; ++i) {
    int s = h.size();
    for (point p : g) {
      while (h.size() >= s + 2 and orient(h[h.size() - 2], h[h.size()
        - 1], p) < lim) {
        h.pop_back();
      }
      h.push_back(p);
    }
    h.pop_back();
    reverse(g.begin(), g.end());
  }
  return h;
}
```

ray

```
float ray_point(point a, point b, point p) {
  if (a != b) {
    line l(a, b);
    if (l.comp(a, p)) return l.dist(p);
  }
  return dist(p, a);
}

float ray_ray(point a, point b, point c, point d) {
  if (a == b) return ray_point(c, d, a);
  if (c == d) return ray_point(a, b, c);
  line l(a, b), r(c, d);
  if (cross(l.v, r.v) != 0) {
    point p = inter(l, r);
    if (l.comp(a, p) and r.comp(c, p)) return 0;
  }
  return min(ray_point(a, b, c), ray_point(c, d, a));
}
```

segment

```
bool in_disk(point a, point b, point p) {
    return dot(a - p, b - p) <= 0;
}

bool on_segment(point a, point b, point p) {
    return orient(a, b, p) == 0 and in_disk(a, b, p);
}

vector<point> proper_inter(point a, point b, point c, point d) {
    T oa = orient(c, d, a),
      ob = orient(c, d, b),
      oc = orient(a, b, c),
      od = orient(a, b, d);
    if (sign(oa) * sign(ob) >= 0 or sign(oc) * sign(od) >= 0) return
      {};
    return {(a * ob - b * oa) / (ob - oa)};
}

vector<point> inters(point a, point b, point c, point d) {
    vector<point> p = proper_inter(a, b, c, d);
    if (p.size()) return p;
    if (on_segment(c, d, a)) p.push_back(a);
    if (on_segment(c, d, b)) p.push_back(b);
    if (on_segment(a, b, c)) p.push_back(c);
    if (on_segment(a, b, d)) p.push_back(d);
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    return p;
}

float segment_point(point a, point b, point p) {
    if (a != b) {
        line l(a, b);
        if (l.comp(a, p) and l.comp(p, b)) return l.dist(p);
    }
    return min(dist(p, a), dist(p, b));
}

float segment_segment(point a, point b, point c, point d) {
    if (proper_inter(a, b, c, d).size()) return 0;
    return min({segment_point(a, b, c), segment_point(a, b, d),
                segment_point(c, d, a), segment_point(c, d, b)});
}
```

graph

binary-lifting

```
struct binary_lifting {
  int n, log;
  vector<int> d;
  vector<vector<int>> up;

  binary_lifting(vector<vector<int>> &g) {
    n = g.size();
    log = __bit_width(n);
    d.assign(n, -1);
    up.assign(log, vector<int>(n));
    for (int s = 0; s < n; ++s) if (d[s] == -1) {
      d[s] = 0;
      up[0][s] = s;
      queue<int> q;
      q.push(s);
      while (q.size()) {
        int a = q.front(); q.pop();
        for (int b : g[a]) if (d[b] == -1) {
          d[b] = d[a] + 1;
          up[0][b] = a;
          q.push(b);
        }
      }
    }
    for (int i = 1; i < log; ++i) {
      for (int a = 0; a < n; ++a) {
        up[i][a] = up[i - 1][up[i - 1][a]];
      }
    }
  }

  int kth(int a, int k) {
    k = min(k, d[a]);
    for (int i = 0; i < log; ++i) {
      if ((k >> i) & 1) a = up[i][a];
    }
    return a;
  }

  int lca(int a, int b) {
    if (d[a] < d[b]) swap(a, b);
    a = kth(a, d[a] - d[b]);
    if (a == b) return a;
    for (int i = log - 1; i >= 0; --i) {
      if (up[i][a] != up[i][b]) {
        a = up[i][a];
        b = up[i][b];
      }
    }
    return up[0][a];
  }

  int dist(int a, int b) {
    return d[a] + d[b] - 2 * d[lca(a, b)];
  }

  int path(int a, int b, int k) {
    int p = lca(a, b);
    int l = d[a] - d[p];
    int r = d[b] - d[p];
    k = min(k, l + r);
    if (k <= l) return kth(a, k);
    else return kth(b, l + r - k);
  }
};
```

centroid-decomposition

```
// Centroid Decomposition: parts[c] = parts left after removing the
  centroid c, as lists of (v, d = distance from v to c)
// Time: O(N log N) | Space: O(N log N)
// Note: centroid_ancestors(parts) gives anc[v] = centroid ancestors
  (c, d) of v, from (v, 0) up to the root
vector<vector<vector<pair<int, int>>>> centroid_decomposition(const
  vector<vector<int>> &g) {
  int n = g.size();
  vector<int> p(n), size(n);
  vector<bool> rem(n);
  vector<vector<vector<pair<int, int>>>> parts(n);
  auto bfs = [&](int r, int h) -> vector<pair<int, int>> {
    vector<pair<int, int>> q = {{r, 1}};
    p[r] = h;
    for (int i = 0; i < q.size(); ++i) {
      auto [a, d] = q[i];
      size[a] = 1;
      for (int b : g[a]) if (b != p[a] and not rem[b]) {
        p[b] = a;
        q.push_back({b, d + 1});
      }
    }
    for (int i = q.size() - 1; i > 0; --i) size[p[q[i].first]] +=
      size[q[i].first];
    return q;
  };
  auto build = [&](auto &self, const vector<pair<int, int>> &q) ->
    void {
    int c = q[0].first;
    for (auto [a, d] : q) if (2 * size[a] > q.size()) c = a;
    rem[c] = true;
    for (int b : g[c]) if (not rem[b]) parts[c].push_back(bfs(b, c));
    for (auto &part : parts[c]) self(self, part);
  };
  for (int a = 0; a < n; ++a) if (not rem[a]) build(build, bfs(a,
    -1));
  return parts;
}

vector<vector<pair<int, int>>> centroid_ancestors(const
  vector<vector<vector<pair<int, int>>>> &parts) {
  int n = parts.size();
  vector<pair<int, int>> ord;
  for (int c = 0; c < n; ++c) {
    int s = 1;
    for (auto &part : parts[c]) s += part.size();
    ord.push_back({s, c});
  }
  sort(ord.begin(), ord.end());
  vector<vector<pair<int, int>>> anc(n);
  for (auto [s, c] : ord) {
    anc[c].push_back({c, 0});
    for (auto &part : parts[c]) for (auto [v, d] : part)
      anc[v].push_back({c, d});
  }
  return anc;
}
```

articulations

connectivity/

```
vector<int> articulations(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n), points;
  vector<bool> art(n);

  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    int c = 0;
    for (int b : g[a]) if (b != p) {
      if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        c++;
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (p != -1 and low[b] >= tin[a]) art[a] = true;
      }
    }
    if (p == -1 and c > 1) art[a] = true;
  };

  for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) dfs(dfs, i, -1);
    if (art[i]) points.push_back(i);
  }
```

```
  return points;
}
```

block-cut-tree

connectivity/

```
// Block-Cut Tree: decomposes an undirected graph into biconnected
  components
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree where nodes [0, N) are original vertices and
  nodes >= N are blocks
vector<vector<int>> block_cut_tree(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n), st;
  vector<vector<int>> bct(n); bct.reserve(2 * n);
  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    st.push_back(a);
    bool parent = true;
    for (int b : g[a]) {
      if (p == b and parent) {
        parent = false;
        continue;
      }
      if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (tin[a] <= low[b]) {
          int i = bct.size();
          bct.emplace_back();
          bct[i].push_back(a);
          bct[a].push_back(i);
          while (true) {
            int c = st.back(); st.pop_back();
            bct[i].push_back(c);
            bct[c].push_back(i);
            if (b == c) break;
          }
        }
      }
    }
  };
  for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) {
      dfs(dfs, i, -1);
    }
  }
  return bct;
}
```

bridge-tree

connectivity/

```
// Bridge Tree: condenses 2-edge-connected components into single
  nodes
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree and an id array where id[a] is the new node
  id of original vertex a
pair<vector<vector<int>>, vector<int>>
  bridge_tree(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n), id(n), st;
  vector<vector<int>> bt;
  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    st.push_back(a);
    bool parent = true;
    for (int b : g[a]) {
      if (b == p and parent) {
        parent = false;
        continue;
      }
      if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
      }
    }
    if (low[a] == tin[a]) {
      int i = bt.size();
      bt.emplace_back();
      while (true) {
        int c = st.back(); st.pop_back();
        id[c] = i;
        if (c == a) break;
      }
    }
  };
  for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) {
      dfs(dfs, i, -1);
    }
    for (int a = 0; a < n; ++a) {
      for (int b : g[a]) {
        if (id[a] != id[b]) {
          bt[id[a]].push_back(id[b]);
        }
      }
    }
    return {bt, id};
}
```

bridges

connectivity/

```
vector<pair<int, int>> bridges(vector<vector<int>> &g) {
  int n = g.size(), t = 0;
  vector<int> tin(n, -1), low(n);
  vector<pair<int, int>> e;

  auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    bool parent = true;
    for (int b : g[a]) {
      if (b == p and parent) {
        parent = false;
      } else if (tin[b] != -1) {
        low[a] = min(low[a], tin[b]);
      } else {
        self(self, b, a);
        low[a] = min(low[a], low[b]);
        if (tin[a] < low[b]) e.emplace_back(a, b);
      }
    }
  };

  for (int i = 0; i < n; ++i) if (tin[i] == -1) dfs(dfs, i, -1);
  return e;
}
```

cycle

connectivity/

```
vector<int> find_cycle(bool directed, vector<vector<int>> &g) {
  int n = g.size();
  vector<int> state(n), parent(n), cycle;

  auto dfs = [&](auto &self, int a, int p) -> bool {
    state[a] = 1;
    bool skip = not directed;
    for (int b : g[a]) {
      if (b == p and skip) {
        skip = false;
      } else if (state[b] == 0) {
        parent[b] = a;
        if (self(self, b, a)) return true;
      } else if (state[b] == 1) {
        for (int c = a; c != b; c = parent[c]) cycle.push_back(c);
        cycle.push_back(b);
        reverse(cycle.begin(), cycle.end());
        return true;
      }
    }
    state[a] = 2;
    return false;
  };

  for (int i = 0; i < n; ++i) if (state[i] == 0 and dfs(dfs, i, -1))
    return cycle;
  return {};
}
```

dominator-tree

connectivity/

```
// Dominator Tree: computes immediate dominators in directed graphs
// Time: build O(V log V + E) | Space: O(V + E)
// Note: returns the tree where node a immediately dominates its
  children
struct dominator_dsu {
  vector<int> p, c;
  vector<int> &x;

  dominator_dsu(int n, vector<int> &x): p(n), c(n), x(x) {
    iota(p.begin(), p.end(), 0);
    iota(c.begin(), c.end(), 0);
  }

  int find(int a) {
    if (p[a] == a) return c[a];
    int r = p[a];
    if (p[r] != r) {
      find(r);
      if (x[c[r]] < x[c[a]]) c[a] = c[r];
      p[a] = p[r];
    }
    return c[a];
  }

  void unite(int a, int b) {
    p[b] = a;
  }
};

vector<vector<int>> dominator_tree(int s, vector<vector<int>> &g) {
  int n = g.size();
  vector<int> tin(n, -1), inv(n), parent(n);
  int t = 0;
  auto dfs = [&](auto &self, int a) -> void {
    tin[a] = t;
    inv[t] = a;
    t++;
    for (int b : g[a]) if (tin[b] == -1) {
      parent[b] = a;
      self(self, b);
    }
  };
  dfs(dfs, s);
  vector<vector<int>> rev(n);
  for (int a = 0; a < n; ++a) {
    for (int b : g[a]) rev[b].push_back(a);
  }
  vector<int> semi(n), dom(n);
  for (int a = 0; a < n; ++a) semi[a] = tin[a];
  vector<vector<int>> bucket(n);
  dominator_dsu dsu(n, semi);
  for (int i = t - 1; i >= 1; --i) {
    int a = inv[i];
    for (int b : rev[a]) if (tin[b] != -1) {
      semi[a] = min(semi[a], semi[dsu.find(b)]);
    }
    bucket[inv[semi[a]]].push_back(a);
    int p = parent[a];
    dsu.unite(p, a);
    for (int b : bucket[p]) {
      dom[b] = p;
      int c = dsu.find(b);
      if (semi[c] < semi[b]) dom[b] = c;
    }
    bucket[p].clear();
  }
  vector<vector<int>> dt(n);
  for (int i = 1; i < t; ++i) {
    int a = inv[i];
    if (dom[a] != inv[semi[a]]) dom[a] = dom[dom[a]];
    dt[dom[a]].push_back(a);
  }
  return dt;
}
```

steiner-tree

connectivity/

```
// Steiner Tree: minimum cost to connect the terminals 0, ..., k - 1
  in a weighted undirected graph
// Time: O(3^k n + 2^k m log m) | Space: O(2^k n)
// Note: steiner[mask][i] = min cost of a tree with the terminals in
  mask and node i
vector<vector<int>> steiner_tree(int k, vector<vector<pair<int,
  int>>> &g) {
  int n = g.size();
  vector<vector<int>> steiner(1 << k, vector<int>(n, inf));
  for (int i = 0; i < n; ++i) steiner[0][i] = 0;
  for (int i = 0; i < k; ++i) steiner[1 << i][i] = 0;
  for (int mask = 1; mask < (1 << k); ++mask) {
    vector<int> &d = steiner[mask];
    for (int sub = (mask - 1) & mask; sub; sub = (sub - 1) & mask) {
      for (int i = 0; i < n; ++i) d[i] = min(d[i], steiner[sub][i] +
        steiner[mask ^ sub][i]);
    }
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>>
      pq;
    for (int i = 0; i < n; ++i) if (d[i] != inf) pq.push({d[i], i});
    while (pq.size()) {
      auto [x, a] = pq.top(); pq.pop();
      if (d[a] < x) continue;
      for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
        d[b] = d[a] + w;
        pq.push({d[b], b});
      }
    }
  }
  return steiner;
}
```

tarjan

connectivity/

```
vector<int> tarjan(const vector<vector<int>> &g) {
  int n = g.size(), timer = 0, m = 0;
  vector<int> in(n, -1), low(n), comp(n, -1);
  vector<int> dfs, edge(n), path;
  for (int s = 0; s < n; ++s) if (in[s] == -1) {
    dfs = {s};
```

```
    while (dfs.size()) {
      int a = dfs.back();
      if (in[a] == -1) {
        in[a] = low[a] = timer++;
        path.push_back(a);
      }
      if (edge[a] < g[a].size()) {
        int b = g[a][edge[a]++];
        if (in[b] == -1) dfs.push_back(b);
        else if (comp[b] == -1) low[a] = min(low[a], in[b]);
        continue;
      }
      dfs.pop_back();
      if (dfs.size()) {
        low[dfs.back()] = min(low[dfs.back()], low[a]);
      }
      if (low[a] == in[a]) {
        while (comp[a] == -1) {
          comp[path.back()] = m;
          path.pop_back();
        }
        m++;
      }
    }
  }
  for (int &c : comp) c = m - 1 - c;
  return comp;
}

vector<vector<int>> condensed_graph(const vector<vector<int>> &g,
  const vector<int> &comp) {
  int m = *max_element(comp.begin(), comp.end()) + 1;
  vector<vector<int>> c(m);
  for (int a = 0; a < g.size(); ++a) {
    for (int b : g[a]) if (comp[a] != comp[b]) {
      c[comp[a]].push_back(comp[b]);
    }
  }
  for (auto &h : c) {
    sort(h.begin(), h.end());
    h.erase(unique(h.begin(), h.end()), h.end());
  }
  return c;
}
```

two-sat

connectivity/

```
// 2-SAT: finds values for n boolean variables that satisfy clauses
  on two literals
// Time: O(n + m) | Space: O(n + m)
// Note: requires tarjan | literal ~a means not a | solve returns the
  values, or empty if impossible
struct two_sat {
  int n;
  vector<vector<int>> g;

  two_sat(int n): n(n), g(2 * n) {}

  int id(int a) {
    return a >= 0 ? 2 * a : 2 * ~a + 1;
  }

  void disjunction(int a, int b) {
    a = id(a), b = id(b);
    g[a ^ 1].push_back(b);
    g[b ^ 1].push_back(a);
  }

  void conjunction(int a, int b) {
    disjunction(a, a), disjunction(b, b);
  }

  void implication(int a, int b) {
    disjunction(~a, b);
  }

  void equal(int a, int b) {
    implication(a, b), implication(b, a);
  }

  void not_equal(int a, int b) {
    disjunction(a, b), disjunction(~a, ~b);
  }

  vector<bool> solve() {
    vector<int> comp = tarjan(g);
    vector<bool> x(n);
    for (int i = 0; i < n; ++i) {
      if (comp[2 * i] == comp[2 * i + 1]) return {};
      x[i] = comp[2 * i] > comp[2 * i + 1];
    }
    return x;
  }
};
```

bellman-ford

distance/

```
vector<int> bellman_ford(int s, int n, vector<array<int, 3>> &e) {
  vector<int> d(n, inf);
  d[s] = 0;
  for (int i = 1; i < n; ++i) {
    bool chd = false;
    for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b]) {
      d[b] = d[a] + w;
      chd = true;
    }
    if (not chd) break;
  }
  for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b])
    return {};
  return d;
}

vector<int> spfa(int s, vector<vector<pair<int, int>>> &g) {
  int n = g.size();
  vector<int> d(n, inf), c(n);
  vector<bool> in(n);
  queue<int> q;
  d[s] = 0;
  in[s] = true;
  q.push(s);
  while (q.size()) {
    int a = q.front(); q.pop();
    in[a] = false;
    for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
      d[b] = d[a] + w;
      if (not in[b]) {
        c[b] += 1;
        if (c[b] >= n) return {};
        in[b] = true;
        q.push(b);
      }
    }
  }
  return d;
}
```

floyd-warshall

distance/

```
vector<vector<int>> floyd_warshall(int n, vector<vector<int>> &g) {
  vector<vector<int>> d = g;
  for (int i = 0; i < n; ++i) {
    d[i][i] = 0;
  }
  for (int k = 0; k < n; ++k) {
    for (int i = 0; i < n; ++i) if (d[i][k] != inf) {
      for (int j = 0; j < n; ++j) if (d[k][j] != inf) {
        d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
      }
    }
  }
  return d;
}
```

johnson

distance/

```
// Johnson: all pairs shortest paths with negative edges, using
  potentials from SPFA and Dijkstra from each vertex
// Time: O(n m log n) | Space: O(n^2 + m)
// Note: requires spfa (bellman-ford) and dijkstra | returns d[i][j]
  (inf if unreachable), or empty on a negative cycle
vector<vector<int>> johnson(vector<vector<pair<int, int>>> g) {
  int n = g.size();
  g.emplace_back();
  for (int i = 0; i < n; ++i) g[n].emplace_back(i, 0);
  vector<int> h = spfa(n, g);
  if (h.empty()) return {};
  g.pop_back();
  for (int i = 0; i < n; ++i) {
    for (auto &[j, w] : g[i]) w += h[i] - h[j];
  }
  vector<vector<int>> d(n);
  for (int i = 0; i < n; ++i) {
    d[i] = dijkstra(i, g);
    for (int j = 0; j < n; ++j) if (d[i][j] != inf) d[i][j] -= h[i] -
      h[j];
  }
  return d;
}
```

potential-method

distance/

Método dos Potenciais

Problema base. O Dijkstra só funciona com pesos ≥ 0. Com arestas negativas (e sem ciclo negativo), queremos trocar os pesos por pesos ≥ 0 **sem mudar quais caminhos são os menores**.

Potencial. É um número $h(u)$ escolhido para cada vértice, como uma "altura". Cada aresta passa a custar o peso original mais o quanto se desce de u para v:

$$w'(u, v) = w(u, v) + h(u) - h(v)$$

Por que os menores caminhos não mudam

Num caminho $s = v_0 \to v_1 \to \cdots \to v_k = t$, os potenciais do meio se cancelam $(+h(v_0) - h(v_1) + h(v_1) - h(v_2) + \cdots - h(v_k))$, sobrando só as pontas:

$$w'(\text{caminho}) = w(\text{caminho}) + h(s) - h(t).$$

- Todo caminho de s a t ganha o **mesmo** valor $h(s) - h(t)$, então o menor continua sendo o menor.
- As distâncias se convertem com $d(s, t) = d'(s, t) - h(s) + h(t)$.
- Um ciclo começa e termina no mesmo vértice, então seu peso não muda.

Como escolher h para ter todo $w' \geq 0$

- $w'(u, v) \geq 0$ é o mesmo que $h(v) \leq h(u) + w(u, v)$ em toda aresta.
- Distâncias mínimas a partir de uma origem já cumprem isso: se $\text{dist}(v) > \text{dist}(u) + w(u, v)$, chegar em v passando por u seria melhor. Então use $h(v) =$ distância mínima até v a partir de um vértice virtual ligado a todos com peso 0 (na prática: Bellman-Ford começando com $h(v) = 0$ para todo v).
- As arestas usadas pelos menores caminhos a partir da origem ficam com $w' = 0$.
- **Ciclo negativo impede:** somando $h(v) - h(u) \leq w(u, v)$ ao longo de um ciclo, o lado esquerdo dá 0, então precisaria $0 \leq w(\text{ciclo})$. Logo existe potencial válido $\iff$ não há ciclo negativo.

dinic

flow/

```
struct dinic {
  struct edge {
    int b, w, i;
    bool h;
  };

  int n;
  vector<vector<edge>> g;
  vector<int> l, p;

  dinic(int n): n(n), g(n), l(n), p(n) {}

  void insert_directed(int a, int b, int w) {
    int i = g[b].size();
    int j = g[a].size();
    g[a].push_back({b, w, i, true});
    g[b].push_back({a, 0, j, false});
  }

  void insert_undirected(int a, int b, int w) {
    int i = g[b].size();
    int j = g[a].size();
    g[a].push_back({b, w, i, true});
    g[b].push_back({a, w, j, true});
  }

  bool bfs(int s, int t) {
    fill(l.begin(), l.end(), -1);
    l[s] = 0;
    queue<int> q;
    q.push(s);

    while (not q.empty()) {
      int a = q.front(); q.pop();
      for (auto &[b, w, i, h] : g[a]) {
        if (w > 0 and l[b] < 0) {
          l[b] = l[a] + 1;
          q.push(b);
        }
      }
    }
    return l[t] >= 0;
  }

  int dfs(int a, int t, int f) {
    if (a == t or f == 0) return f;

    for (int &k = p[a]; k < g[a].size(); ++k) {
      auto &[b, w, i, h] = g[a][k];
      if (w > 0 and l[b] == l[a] + 1) {
        if (int d = dfs(b, t, min(f, w))) {
          w -= d;
          g[b][i].w += d;
          return d;
```

```
      }
    }
  }
  return 0;
}

int flow(int s, int t) {
  int f = 0;
  while (bfs(s, t)) {
    fill(p.begin(), p.end(), 0);
    while (int d = dfs(s, t, inf)) {
      f += d;
    }
  }
  return f;
}

bool on_cut(int a) {
  return l[a] >= 0;
}

vector<pair<int, int>> saturated_edges() {
  vector<pair<int, int>> e;
  for (int a = 0; a < n; ++a) {
    for (auto [b, w, i, h] : g[a]) if (h and w == 0) {
      e.push_back({a, b});
    }
  }
  return e;
}
};
```

flow-theorems

flow/

Teoremas de Fluxo

Notação. S é a fonte, T é o sumidouro, $c(u, v)$ é a capacidade e maxflow(S, T) é o fluxo máximo. O **residual** tem a aresta $u \to v$ se ainda cabe fluxo em $u \to v$ ou se há fluxo em $v \to u$ (que pode ser desfeito). Depois do fluxo máximo, A é o conjunto de vértices que S alcança no residual.

Base

- **Corte mínimo:** maxflow(S, T) é a menor soma de capacidades de arestas que, removidas, separam S de T. O corte são as arestas originais $u \to v$ com $u \in A$ e $v \notin A$.
- Pode haver vários cortes mínimos. Uma aresta $u \to v$ está em **todo** corte mínimo (aumentar $c(u, v)$ aumenta o fluxo máximo) $\iff$ $u \in A$ e T é alcançável a partir de v no residual.
- Ela está em **algum** corte mínimo (diminuir $c(u, v)$ diminui o fluxo máximo) $\iff$ $u \to v$ está saturada e v não é alcançável a partir de u no residual.
- **Várias fontes ou sumidouros:** $S \to$ cada fonte e cada sumidouro $\to T$, com ∞.
- **Aresta não dirigida:** $u \to v$ e $v \to u$, ambas com c.

Reduções clássicas

- **Caminhos disjuntos em arestas:** capacidade 1 em cada aresta. O fluxo é o maior número de caminhos de S a T sem aresta em comum, e também o menor número de arestas que separam S de T.
- **Capacidade em vértice:** divida u em $u_{in} \to u_{out}$ com capacidade $c(u)$; as arestas que entram vão para u_{in} e as que saem partem de u_{out}. Com $c(u) = 1$, dá caminhos disjuntos em vértices. Com $c(u) =$ peso de u e ∞ nas arestas, o corte é o conjunto de vértices de peso mínimo que separa S de T.
- **Project selection:** escolher u obriga escolher v. Lucro $p_u > 0$: $S \to u$ com p_u; custo $c_u > 0$: $u \to T$ com c_u; dependência: $u \to v$ com ∞. Lucro máximo $= \sum p_u -$ corte mínimo, e os escolhidos são $A \setminus \{S\}$.
- **Escolha entre dois lados:** cada u fica do lado de S (custo b_u) ou de T (custo a_u): $S \to u$ com a_u e $u \to T$ com b_u. Penalidade c se u e v ficam em lados diferentes: $u \to v$ e $v \to u$, ambas com c. O custo mínimo é o corte mínimo.

Duas fontes e dois sumidouros

Em grafo **não dirigido**, o fluxo de s_1 tem que chegar em t_1 (d_1 unidades) e o de s_2 em t_2 (d_2 unidades), sem misturar. Rode dois fluxos, cada um com uma fonte S e um sumidouro T novos:

- $S \to s_1$ e $t_1 \to T$ com d_1; $S \to s_2$ e $t_2 \to T$ com d_2.
- O mesmo, mas com s_2 e t_2 trocados: $S \to t_2$ e $s_2 \to T$ com d_2.
- É possível $\iff$ os dois fluxos dão $d_1 + d_2$. Se todas as capacidades são pares, dá com fluxo inteiro; senão, pode precisar de meias unidades. Em grafo dirigido isso não vale.

DAG com pesos nos vértices

- **Anticadeia de peso máximo:** no fecho transitivo, ligue $S \to u_L$ com w_u, $u_L \to v_R$ com ∞ se u alcança v, e $v_R \to T$ com w_v. O peso máximo é $\sum w -$ maxflow(S, T), e a anticadeia são os v com $v_L \in A$ e $v_R \notin A$.
- **Dilworth com pesos:** esse valor também é o menor número de caminhos (podendo repetir vértices) em que cada v aparece pelo menos w_v vezes. Para montar os caminhos, divida v em $v_{in} \to v_{out}$ com limite inferior w_v, ligue $S \to v_{in}$, $v_{out} \to T$ e $u_{out} \to v_{in}$ para cada aresta, tudo com ∞, e calcule o fluxo mínimo.

Limite inferior nas arestas ($l \le f \le r$)

Mande os l obrigatórios em cada aresta e corrija o desequilíbrio que isso cria nos vértices.

- Cada aresta $u \to v$ vira $u \to v$ com capacidade $r - l$, e o saldo muda: $d[v] \mathrel{+}= l$, $d[u] \mathrel{-}= l$.
- Adicione $T \to S$ com ∞, uma fonte nova S' e um sumidouro novo T': $S' \to v$ com $d[v]$ se $d[v] > 0$, e $v \to T'$ com $-d[v]$ se $d[v] < 0$.
- **Viável** $\iff$ maxflow$(S', T') = \sum_{d[v]>0} d[v]$. O fluxo real de cada aresta é l mais o fluxo nela na rede nova.
- **Fluxo máximo de S a T:** seja f_0 o fluxo na aresta $T \to S$. Remova essa aresta e rode no residual: resposta $= f_0 +$ maxflow(S, T).
- **Fluxo mínimo:** igual, mas a resposta é $f_0 -$ maxflow(T, S).

Gomory-Hu

Em grafo não dirigido, $n - 1$ fluxos montam uma árvore em que o corte mínimo entre u e v é a menor aresta do caminho entre eles. Comece com $p[i] = 0$ para todo i. Para $i = 1, \ldots, n - 1$: calcule o corte entre i e $p[i]$, com valor c e lado A de i; crie a aresta $(i, p[i], c)$; e, para todo $j > i$ com $p[j] = p[i]$ e $j \in A$, faça $p[j] = i$.

min-cost-k-flow

flow/

```
// Min Cost K-Flow: sends up to k units from s to t with minimum
  total cost, by successive shortest paths
// Time: O(F E log V) | Space: O(V + E)
// Note: if all costs are non-negative, replace spfa with
  fill(h.begin(), h.end(), 0)
struct min_cost_k_flow {
  struct edge {
    int b, i, w, c;
    bool o;
  };

  int n;
  vector<vector<edge>> g;
  vector<int> d, h, p, e;

  min_cost_k_flow(int n): n(n), g(n), d(n), h(n), p(n), e(n) {}

  void insert_edge(int a, int b, int w, int c) {
    int i = g[b].size(), j = g[a].size();
    g[a].push_back({b, i, w, c, true});
    g[b].push_back({a, j, 0, -c, false});
  }

  bool dijkstra(int s, int t) {
    fill(d.begin(), d.end(), inf);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>>
      pq;
    d[s] = 0;
    pq.push({0, s});
    while (pq.size()) {
      auto [x, a] = pq.top(); pq.pop();
      if (d[a] < x) continue;
      for (int i = 0; i < g[a].size(); ++i) {
        auto [b, j, w, c, o] = g[a][i];
        int v = d[a] + c + h[a] - h[b];
        if (w == 0 or v >= d[b]) continue;
        d[b] = v, p[b] = a, e[b] = i;
        pq.push({v, b});
      }
    }
    return d[t] != inf;
  }

  pair<int, int> cost_flow(int s, int t, int k = inf) {
    vector<vector<pair<int, int>>> adj(n);
    for (int a = 0; a < n; ++a) {
      for (auto [b, i, w, c, o] : g[a]) if (w > 0) {
        adj[a].push_back({b, c});
      }
    }
    h = spfa(s, adj);
    if (h.empty()) return {-1, -1};
    int flow = 0, cost = 0;
    while (flow < k and dijkstra(s, t)) {
      for (int a = 0; a < n; ++a) if (d[a] != inf) h[a] += d[a];
      int push = k - flow;
      for (int a = t; a != s; a = p[a]) push = min(push,
        g[p[a]][e[a]].w);
      for (int a = t; a != s; a = p[a]) {
        edge &x = g[p[a]][e[a]];
        x.w -= push, g[a][x.i].w += push;
        cost += push * x.c;
      }
      flow += push;
    }
    return {flow, cost};
  }

  vector<pair<int, int>> saturated_edges() {
    vector<pair<int, int>> edges;
    for (int a = 0; a < n; ++a) {
      for (auto [b, i, w, c, o] : g[a]) if (w == 0 and o)
        edges.push_back({a, b});
    }
    return edges;
  }
};
```

hierholzer

```
// Eulerian Path: Hierholzer's algorithm for paths and circuits, in
  directed or undirected graphs
// Time: O(V + E) | Space: O(V + E)
// Note: s and t are the first and last vertices, -1 for any |
  returns the vertices, or empty if there is no such path
vector<int> eulerian_path(bool directed, vector<vector<pair<int,
  int>>> &g, int s = -1, int t = -1) {
  int n = g.size(), m = 0, k = 0;
  vector<int> d(n);
  for (int a = 0; a < n; ++a) {
    m += g[a].size();
    for (auto [b, id] : g[a]) {
      if (directed) d[a]++, d[b]--;
      else d[a] ^= 1;
    }
  }
  if (not directed) m /= 2;
  for (int a = 0; a < n; ++a) {
    if (abs(d[a]) > 1) return {};
    if (d[a] == 1) k++;
  }

  for (int a = 0; a < n; ++a) if (s == -1 and d[a] == 1 and a != t) s
    = a;
  if (s == -1) s = t;
  for (int a = 0; a < n; ++a) if (s == -1 and g[a].size()) s = a;
  if (s == -1) s = 0;
  if (k > 2 or (directed and k > 1) or (k > 0 and d[s] != 1)) return
    {};

  vector<bool> used(m + 1);
  vector<int> next(n), path, dfs = {s};
  while (dfs.size()) {
    int a = dfs.back();
    if (next[a] == g[a].size()) {
      path.push_back(a), dfs.pop_back();
    } else {
      auto [b, id] = g[a][next[a]++];
      if (not used[id]) used[id] = true, dfs.push_back(b);
    }
  }
  if (path.size() != m + 1) return {};
  reverse(path.begin(), path.end());
  if (t != -1 and path.back() != t) return {};
  return path;
}
```

hld

```
// Heavy-Light Decomposition: splits a path into O(log N) ranges of
  positions for an external structure
// Time: O(N) build, O(log N) per path | Space: O(N)
// Usage: hld h(g, false) values on vertices, hld h(g, true) values
  on edges | vertex v is at h.id[v], edge u-v at h.edge(u, v)
struct hld {
  int t;
  bool e;
  vector<int> p, h, s, id;

  hld(vector<vector<int>> &g, bool e): t(0), e(e) {
    int n = g.size();
    p.resize(n);
    h.resize(n);
    s.resize(n);
    id.resize(n);
    for (int i = 0; i < n; ++i) if (s[i] == 0) {
      p[i] = h[i] = i;
      dfs(i, g);
      decompose(i, g);
    }
  }
```

```
  void dfs(int a, vector<vector<int>> &g) {
    s[a] = 1;
    for (int &b : g[a]) if (b != p[a]) {
      p[b] = a;
      dfs(b, g);
      s[a] += s[b];
      if (g[a][0] == p[a] or s[b] > s[g[a][0]]) {
        swap(b, g[a][0]);
      }
    }
  }

  void decompose(int a, vector<vector<int>> &g) {
    id[a] = t++;
    for (int b : g[a]) if (b != p[a]) {
      h[b] = (b == g[a][0] ? h[a] : b);
      decompose(b, g);
    }
  }

  int edge(int u, int v) {
    return max(id[u], id[v]);
  }

  vector<tuple<int, int, bool>> path(int a, int b) {
    vector<tuple<int, int, bool>> up, down;
    while (h[a] != h[b]) {
      if (id[h[a]] > id[h[b]]) {
        up.push_back({id[h[a]], id[a], true});
        a = p[h[a]];
      } else {
        down.push_back({id[h[b]], id[b], false});
        b = p[h[b]];
      }
    }
    if (id[a] > id[b]) {
      up.push_back({id[b] + e, id[a], true});
    } else if (id[a] + e <= id[b]) {
      down.push_back({id[a] + e, id[b], false});
    }
    up.insert(up.end(), down.rbegin(), down.rend());
    return up;
  }

  tuple<int, int> subtree(int v) {
    return {id[v] + e, id[v] + s[v] - 1};
  }
};
```

leaf-peeling

```
// Leaf Peeling: finds the center(s) and diameter of an undirected
  tree
// Time: O(N) | Space: O(N)
// Note: returns {diameter, centers} | root is the first vertex, 0 or
  1
pair<int, vector<int>> leaf_peeling(int root, vector<vector<int>> &g)
  {
  int n = g.size() - root, layers = 0;
  if (n == 1) return {0, {root}};
  vector<int> d(g.size()), leaves, next;
  for (int a = root; a < g.size(); ++a) {
    d[a] = g[a].size();
    if (d[a] == 1) leaves.push_back(a);
  }
  while (n > 2) {
    n -= leaves.size(), layers++;
    next.clear();
    for (int a : leaves) for (int b : g[a]) if (--d[b] == 1)
      next.push_back(b);
    swap(leaves, next);
  }
  return {2 * layers + (leaves.size() == 2), leaves};
}
```

link-cut-tree

```
// Link-Cut Tree: dynamic forest with link, cut, connected, lca, path
  query, path update and set
// Time: O(log N) amortized per operation | Space: O(N)
// Note: vertices are 1-based (a[0] is ignored) | lca is 0 if not
  connected | edit node, pull and apply
struct link_cut_tree {
  struct node {
    int val = 0, size = 0;
    int sum = 0, lazy = 0;
    bool rev = false;
    int p = 0;
    array<int, 2> c = {0, 0};
  };

  void pull(int n) {
    node &a = tree[n], &l = tree[a.c[0]], &r = tree[a.c[1]];
    a.size = l.size + 1 + r.size;
    a.sum = l.sum + a.val + r.sum;
  }

  void apply(int n, int x) {
    if (n == 0) return;
    tree[n].val += x;
    tree[n].sum += x * tree[n].size;
    tree[n].lazy += x;
  }

  vector<node> tree;

  link_cut_tree(const vector<int> &a): tree(a.size()) {
    for (int i = 1; i < a.size(); ++i) {
      tree[i].val = a[i];
      pull(i);
    }
  }

  void flip(int n) {
    if (n == 0) return;
    swap(tree[n].c[0], tree[n].c[1]);
    tree[n].rev ^= 1;
  }

  void push(int n) {
    if (tree[n].rev) flip(tree[n].c[0]), flip(tree[n].c[1]),
      tree[n].rev = false;
    if (tree[n].lazy != tree[0].lazy) {
      apply(tree[n].c[0], tree[n].lazy), apply(tree[n].c[1],
        tree[n].lazy);
      tree[n].lazy = tree[0].lazy;
    }
  }

  bool is_root(int n) {
    int p = tree[n].p;
    return tree[p].c[0] != n and tree[p].c[1] != n;
  }

  void rotate(int n) {
    int p = tree[n].p, g = tree[p].p, d = tree[p].c[1] == n;
    if (not is_root(p)) tree[g].c[tree[g].c[1] == p] = n;
    tree[n].p = g;
    tree[p].c[d] = tree[n].c[d ^ 1];
    if (tree[n].c[d ^ 1]) tree[tree[n].c[d ^ 1]].p = p;
    tree[n].c[d ^ 1] = p;
    tree[p].p = n;
    pull(p), pull(n);
  }

  void splay(int n) {
    static vector<int> s;
    s = {n};
    for (int a = n; not is_root(a); a = tree[a].p)
      s.push_back(tree[a].p);
    while (s.size()) push(s.back()), s.pop_back();
    while (not is_root(n)) {
      int p = tree[n].p, g = tree[p].p;
      if (not is_root(p)) rotate((tree[g].c[1] == p) == (tree[p].c[1]
        == n) ? p : n);
      rotate(n);
    }
  }

  int access(int a) {
    int last = 0;
    for (int x = a; x != 0; x = tree[x].p) {
      splay(x);
      tree[x].c[1] = last;
      pull(x);
      last = x;
    }
    splay(a);
    return last;
  }

  void make_root(int a) {
    access(a);
    flip(a);
  }

  int find_root(int a) {
    access(a);
    while (tree[a].c[0] != 0) push(a), a = tree[a].c[0];
    splay(a);
    return a;
  }

  void link(int a, int b) {
    make_root(a);
    if (find_root(b) != a) tree[a].p = b;
  }

  void cut(int a, int b) {
    make_root(a);
    access(b);
    if (tree[b].c[0] == a and tree[a].c[1] == 0) {
      tree[b].c[0] = tree[a].p = 0;
      pull(b);
    }
  }

  bool connected(int a, int b) {
    return find_root(a) == find_root(b);
  }

  int lca(int a, int b) {
    if (not connected(a, b)) return 0;
    access(a);
    return access(b);
  }

  void set(int a, int x) {
    access(a);
    tree[a].val = x;
    pull(a);
  }

  void update(int a, int b, int x) {
    make_root(a);
    access(b);
    apply(b, x);
  }

  node query(int a, int b) {
    make_root(a);
    access(b);
    return tree[b];
  }
};
```

blossom

matching/

```
// Blossom: maximum matching in a general graph, with Edmonds'
  algorithm
// Time: O(V^3) | Space: O(V + E)
// Note: undirected graph, an edge goes in both lists | returns
  match[a], the vertex matched with a or -1
vector<int> blossom(vector<vector<int>> &g) {
  int n = g.size();
  vector<int> match(n, -1), p(n), base(n), q;
  vector<bool> used(n), mark(n);

  auto lca = [&](int a, int b) -> int {
    vector<bool> seen(n);
    for (a = base[a]; ; a = base[p[match[a]]]) {
      seen[a] = true;
      if (match[a] == -1) break;
    }
    for (b = base[b]; not seen[b]; b = base[p[match[b]]]);
    return b;
  };

  auto mark_path = [&](int a, int c, int child) {
    while (base[a] != c) {
      mark[base[a]] = mark[base[match[a]]] = true;
      p[a] = child, child = match[a], a = p[match[a]];
    }
  };

  auto find_path = [&](int root) -> int {
    fill(used.begin(), used.end(), false);
    fill(p.begin(), p.end(), -1);
    iota(base.begin(), base.end(), 0);
    used[root] = true, q = {root};
    for (int h = 0; h < q.size(); ++h) {
      int a = q[h];
      for (int b : g[a]) {
        if (base[a] == base[b] or match[a] == b) continue;
        if (b == root or (match[b] != -1 and p[match[b]] != -1)) {
          int c = lca(a, b);
          fill(mark.begin(), mark.end(), false);
          mark_path(a, c, b), mark_path(b, c, a);
          for (int i = 0; i < n; ++i) if (mark[base[i]]) {
            base[i] = c;
            if (not used[i]) used[i] = true, q.push_back(i);
          }
        } else if (p[b] == -1) {
          p[b] = a;
          if (match[b] == -1) return b;
          used[match[b]] = true, q.push_back(match[b]);
```

```
        }
      }
    }
    return -1;
  };

  for (int a = 0; a < n; ++a) {
    for (int b : g[a]) if (a != b and match[a] == -1 and match[b] ==
      -1) match[a] = b, match[b] = a;
  }
  for (int a = 0; a < n; ++a) {
    if (match[a] != -1) continue;
    for (int b = find_path(a); b != -1;) {
      int c = p[b], d = match[c];
      match[b] = c, match[c] = b, b = d;
    }
  }
  return match;
}
```

hopcroft-karp

matching/

```
// Hopcroft-Karp: maximum bipartite matching, augmenting along
  shortest paths in phases
// Time: O(E sqrt(V)) | Space: O(V + E)
// Note: g[a] has the right neighbors of left vertex a | returns
  match[b], the left vertex of b or -1, for b up to the largest right
  vertex
vector<int> hopcroft_karp(vector<vector<int>> &g) {
  int n = g.size(), m = 0, lim = -1;
  for (int a = 0; a < n; ++a) for (int b : g[a]) m = max(m, b + 1);
  vector<int> match(m, -1), d(n), itr(n);
  vector<bool> used(n);

  auto bfs = [&]() -> bool {
    queue<int> q;
    for (int a = 0; a < n; ++a) {
      d[a] = -1;
      if (not used[a]) d[a] = 0, q.push(a);
    }
    lim = -1;
    while (q.size()) {
      int a = q.front(); q.pop();
      if (lim != -1 and d[a] >= lim) continue;
      for (int b : g[a]) {
        int c = match[b];
        if (c == -1 and lim == -1) lim = d[a] + 1;
        if (c != -1 and d[c] == -1) d[c] = d[a] + 1, q.push(c);
      }
    }
    return lim != -1;
  };

  auto dfs = [&](auto &self, int a) -> bool {
    for (int &i = itr[a]; i < g[a].size(); ++i) {
      int b = g[a][i], c = match[b];
      if (c == -1 and d[a] + 1 != lim) continue;
      if (c == -1 or (d[c] == d[a] + 1 and self(self, c))) return
        used[a] = true, match[b] = a, true;
    }
    return d[a] = -1, false;
  };

  while (bfs()) {
    fill(itr.begin(), itr.end(), 0);
    for (int a = 0; a < n; ++a) if (not used[a]) dfs(dfs, a);
  }
  return match;
}
```

hungarian

matching/

```
// Hungarian Algorithm: minimum weight bipartite matching
// Time: O(N^2 * M) | Space: O(N + M)
// Note: computes min cost assignment | negate input matrix for max
pair<int, vector<int>> hungarian(vector<vector<int>> g) {
  int n = g.size(), m = g[0].size();
  bool transposed = false;
  if (n > m) {
    transposed = true;
    vector<vector<int>> t(m, vector<int>(n));
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < m; ++j) {
        t[j][i] = g[i][j];
      }
    }
    swap(n, m);
    swap(g, t);
  }

  vector<int> pot_row(n + 1), pot_col(m + 1);
  vector<int> match(m + 1), prev_col(m + 1);

  for (int i = 1; i <= n; ++i) {
    match[0] = i;
    int col = 0;
    vector<int> min_cost(m + 1, inf);
    vector<bool> v(m + 1);
    do {
      v[col] = true;
      int row = match[col];
      int delta = inf;
      int next_col = 0;
      for (int j = 1; j <= m; ++j) if (not v[j]) {
        int cost = g[row - 1][j - 1] - pot_row[row] - pot_col[j];
        if (cost < min_cost[j]) {
          min_cost[j] = cost;
          prev_col[j] = col;
        }
        if (min_cost[j] < delta) {
          delta = min_cost[j];
          next_col = j;
        }
      }
      for (int j = 0; j <= m; ++j) {
        if (v[j]) {
          pot_row[match[j]] += delta;
          pot_col[j] -= delta;
        } else {
          min_cost[j] -= delta;
        }
      }
      col = next_col;
    } while (match[col] != 0);
    do {
      int prev = prev_col[col];
      match[col] = match[prev];
      col = prev;
    } while (col != 0);
  }

  int cost = -pot_col[0];
  vector<int> answer(n, -1);
  for (int j = 1; j <= m; ++j) if (match[j] != 0) {
    answer[match[j] - 1] = j - 1;
  }

  if (transposed) {
    vector<int> real(m, -1);
    for (int i = 0; i < n; ++i) if (answer[i] != -1) {
      real[answer[i]] = i;
    }
    swap(answer, real);
  }
  return {cost, answer};
}
```

kuhn

matching/

```
// Kuhn's Algorithm: maximum bipartite matching
// Time: O(V E), usually much faster | Space: O(V)
// Note: g[a] has the right neighbors of left vertex a | returns
  match[b], the left vertex of b or -1, for b up to the largest right
  vertex
vector<int> kuhn(vector<vector<int>> &g) {
  int n = g.size(), m = 0, itr = 0;
  for (int a = 0; a < n; ++a) for (int b : g[a]) m = max(m, b + 1);
  vector<int> match(m, -1), vis(n, -1);
  vector<bool> used(n);

  auto dfs = [&](auto &self, int a) -> bool {
    if (vis[a] == itr) return false;
    vis[a] = itr;
    for (int b : g[a]) if (match[b] == -1) return match[b] = a, true;
    for (int b : g[a]) if (self(self, match[b])) return match[b] = a,
      true;
    return false;
  };

  for (bool found = true; found; ++itr) {
    found = false;
    for (int a = 0; a < n; ++a) if (not used[a] and dfs(dfs, a))
      used[a] = found = true;
  }
  return match;
}
```

matching-theorems

matching/

Teoremas de Matching

Notação. M é um matching máximo, $|M|$ é o seu tamanho e $|V|$ é o número de vértices. Em bipartido, $V = L \cup R$. Um vértice é **livre** se nenhuma aresta de M toca nele.

Em bipartidos

- **König:** cobertura mínima de vértices (MVC), o menor conjunto de vértices que toca toda aresta, tem $|\text{MVC}| = |M|$.
- **Conjunto independente máximo (MIS):** o complemento de uma cobertura, $|\text{MIS}| = |V| - |M|$.
- **Cobertura mínima de arestas** (sem vértices isolados): $|V| - |M|$. Pegue o matching e adicione uma aresta qualquer de cada vértice livre.
- **Hall:** existe matching que casa todo $L \iff |N(X)| \geq |X|$ para todo $X \subseteq L$, onde $N(X)$ são os vizinhos de X. Em geral, $|M| = |L| - \max_{X \subseteq L}(|X| - |N(X)|)$, com $X = \emptyset$ valendo 0.
- **Pesos nos vértices:** ligue $S \to u$ com w_u, $u \to v$ com ∞ e $v \to T$ com w_v ($u \in L$, $v \in R$). A MVC de peso mínimo é o corte mínimo, e o MIS de peso máximo é $\sum w -$ corte.

Recuperar a resposta

Com fluxo ($S \to L \to R \to T$, capacidade 1), seja Z o que a BFS de S alcança no residual. Sem fluxo, oriente as arestas fora de M de L para R e as de M de R para L, e seja Z o que se alcança a partir dos vértices livres de L.

- **Matching:** arestas $L \to R$ com fluxo 1.
- **MVC:** $(L \setminus Z) \cup (R \cap Z)$.
- **MIS:** o complemento, $(L \cap Z) \cup (R \setminus Z)$.

Cobertura de DAG por caminhos

Divida cada vértice u em u_L e u_R e, para cada aresta $u \to v$, ligue $u_L \to v_R$. Cada aresta do matching junta dois vértices no mesmo caminho.

- **Sem repetir vértices:** o mínimo de caminhos é $|V| - |M|$.
- **Repetindo vértices:** faça o mesmo no fecho transitivo ($u \to v$ se u alcança v). O mínimo $|V| - |M|$ é igual à maior anticadeia, o maior conjunto em que ninguém alcança ninguém (Dilworth).
- **Exemplo:** $a \to c$, $b \to c$, $c \to d$, $c \to e$ precisa de 3 caminhos sem repetir, mas de 2 repetindo (acd e bce).
- **Maior anticadeia:** os v com v_L e v_R fora da MVC do grafo dividido (no fecho).
- **Mirsky:** o menor número de anticadeias que cobre todos os vértices é o tamanho da maior cadeia (o caminho mais longo, contando vértices).
- **Caminhos:** siga as arestas $u_L \to v_R$ do matching; u termina um caminho se u_L está livre.

Em qualquer grafo

- **Gallai:** $|\text{MIS}| + |\text{MVC}| = |V|$ e, sem vértices isolados, $|M| + |\text{cobertura mínima de arestas}| = |V|$.
- König só vale em bipartido: no triângulo, $|\text{MVC}| = 2$ e $|M| = 1$.

Modelagens

- **Torres:** linhas de um lado, colunas do outro e uma aresta para cada casa (i, j) permitida. O máximo de torres sem ataque é $|M|$, e o mínimo de linhas e colunas que cobrem todas as casas também (König).

matroid-intersection

matching/

Interseção de Matroides

Matroide: uma regra que diz quais conjuntos de elementos são **válidos**. O vazio é válido, tirar elementos de um válido continua válido, e um válido menor sempre pode crescer com algum elemento de um válido maior. Os válidos máximos têm todos o mesmo tamanho, o **posto** r.

Matroides comuns

- **Floresta:** arestas sem ciclo. Testa com DSU.
- **Cores:** no máximo c_i elementos da cor i. Testa contando.
- **Vetores:** linearmente independentes (ou em xor). Testa com base xor.

Para que serve

Achar o maior conjunto válido nos dois matroides ao mesmo tempo. Exemplos:

- **Emparelhamento bipartido:** cada vértice da esquerda no máximo uma vez, e cada vértice da direita no máximo uma vez.
- **Árvore geradora com no máximo c_i arestas da cor i:** floresta e cores. Existe se a resposta tem $n - 1$ arestas.
- **Base xor com limite por cor:** vetores e cores.
- **Arborescência com raiz s:** floresta (sem olhar a direção) e no máximo uma aresta chegando em cada vértice, nenhuma chegando em s.

Algoritmo

Comece com S vazio e, a cada rodada, tente aumentar S em um elemento. Não basta colocar um elemento que caiba nos dois matroides: às vezes é preciso tirar alguns de S e colocar outros no lugar, como num caminho aumentante de

emparelhamento.

1. **Grafo de trocas.** Os vértices são os elementos. Para y em S e x fora de S:
   - $y \to x$ se $S - y + x$ é válido no matroide 1 (x pode ocupar o lugar de y no matroide 1);
   - $x \to y$ se $S - y + x$ é válido no matroide 2 (x pode ocupar o lugar de y no matroide 2).
2. **Origens e destinos.** As origens são os x fora de S que cabem em S no matroide 1 ($S + x$ válido nele), e os destinos são os que cabem no matroide 2. Um x que é as duas coisas entra direto.
3. **Caminho.** Faça uma BFS que começa em todas as origens ao mesmo tempo e pare no primeiro destino retirado da fila. O caminho fica $x_0 \to y_1 \to x_1 \to \cdots \to y_k \to x_k$, alternando quem está fora e quem está em S.
4. **Troca.** Os x entram em S e os y saem: entram $k+1$ e saem k, então S cresce em 1. No matroide 1, x_0 entra numa vaga livre e cada x_i ocupa o lugar de y_i. No matroide 2, cada x_{i-1} ocupa o lugar de y_i e x_k entra numa vaga livre.
5. **Por que o mais curto.** Cada troca é válida sozinha, mas todas juntas só são garantidas se o caminho não tem atalho. O caminho da BFS nunca tem atalho.

Quando não há caminho, S é o maior possível. São até r rodadas, cada uma com $O(r \cdot |E|)$ testes.

Testes rápidos. Antes de cada rodada, para cada y em S, monte uma vez a estrutura de S sem y e teste todos os x nela:

- **Floresta:** DSU com as arestas de $S - y$; $x = (u, v)$ cabe se u e v estão em componentes diferentes.
- **Cores:** x cabe se a sua cor i tem menos de c_i elementos em $S - y$.
- **Vetores:** base xor de $S - y$; x cabe se não é gerado por ela.

Fórmula de Edmonds

Para um conjunto A de elementos, $r_i(A)$ é o tamanho do maior válido dentro de A no matroide i. Então:

$$\text{resposta} = \min_{A} \; r_1(A) + r_2(E \setminus A).$$

- **Por que vale $\leq$:** um S válido nos dois tem no máximo $r_1(A)$ elementos dentro de A e no máximo $r_2(E \setminus A)$ fora dele.
- **Quem atinge o mínimo:** quando o algoritmo para, $A =$ os elementos que a BFS **não** alcançou.
- **Uso:** problemas que pedem o mínimo de uma soma de postos, ou provar que a resposta não pode ser maior.

Com pesos

Cada elemento tem peso w, e queremos o maior peso de um S válido nos dois com $|S| = k$, para cada k.

- **Mesmo grafo de trocas,** mas cada vértice tem custo: $-w(x)$ para quem está fora (vai entrar) e $+w(y)$ para quem está em S (vai sair). O custo do caminho é a soma dos custos dos vértices, inclusive as pontas.
- **Caminho mais barato** das origens até os destinos. Entre os de mesmo custo, o de menos vértices. Use Bellman-Ford ou SPFA, porque há custos negativos (mas nunca ciclo negativo).
- **Garantia:** depois de cada troca, S é o de maior peso entre todos com o mesmo tamanho. Para o maior peso de qualquer tamanho, guarde o melhor S ao longo das rodadas.

max-clique

```cpp
// Max Clique: Bron-Kerbosch with the pivot of most neighbors and
  pruning by the best size
// Time: O(3^(n / 3)) | Space: O(n)
// Note: for a maximum independent set, run it on the complement graph
vector<int> max_clique(vector<vector<int>> &g) {
  auto size = [](int x) -> int { return __builtin_popcountll(x); };
  auto first = [](int x) -> int { return __builtin_ctzll(x); };

  int n = g.size(), all = 0, best = 0;
  vector<int> adj(n);
  for (int a = 0; a < n; ++a) {
    all |= 1LL << a;
    for (int b : g[a]) if (b != a) {
      adj[a] |= 1LL << b;
    }
  }

  auto rec = [&](auto &self, int clique, int cand) -> void {
    if (size(clique) + size(cand) <= size(best)) return;
    if (cand == 0) return void(best = clique);
    int pivot = first(cand);
    for (int rest = cand; rest; rest &= rest - 1) {
      int v = first(rest);
      if (size(cand & adj[v]) > size(cand & adj[pivot])) pivot = v;
    }
    for (int rest = cand & ~adj[pivot]; rest; rest &= rest - 1) {
      int v = first(rest);
      self(self, clique | (1LL << v), cand & adj[v]);
      cand &= ~(1LL << v);
    }
  };
  rec(rec, 0, all);

  vector<int> res;
  for (int v = 0; v < n; ++v) if ((best >> v) & 1) res.push_back(v);
  return res;
}
```

virtual-tree

```cpp
// Virtual Tree: compresses the tree to the given vertices and their
  lcas, at most 2k - 1 vertices
// Time: O(N log N) build, O(1) lca, O(k log k) per compression |
  Space: O(N log N)
// Note: build(v) returns {u, p} sorted by tin, with p = -1 at the
  root of the compressed tree | edge p-u is a path of d[u] - d[p] edges
struct virtual_tree {
  vector<int> tin, d;
  vector<vector<int>> st;

  virtual_tree(vector<vector<int>> &g, int root): tin(g.size()),
    d(g.size()) {
    int n = g.size(), t = 0;
    st.assign(__lg(n) + 1, vector<int>(n));
    auto dfs = [&](auto &self, int a, int p) -> void {
      tin[a] = t, st[0][t++] = p;
      for (int b : g[a]) if (b != p) d[b] = d[a] + 1, self(self, b,
        a);
    };
    dfs(dfs, root, root);
    for (int j = 1; j < st.size(); ++j) {
      for (int i = 0; i + (1 << j) <= n; ++i) st[j][i] = first(st[j -
        1][i], st[j - 1][i + (1 << (j - 1))]);
    }
  }

  int first(int a, int b) {
    return tin[a] < tin[b] ? a : b;
  }

  int lca(int a, int b) {
    if (a == b) return a;
    if (tin[a] > tin[b]) swap(a, b);
    int l = tin[a] + 1, r = tin[b] + 1, j = __lg(r - l);
    return first(st[j][l], st[j][r - (1 << j)]);
  }

  int dist(int a, int b) {
    return d[a] + d[b] - 2 * d[lca(a, b)];
  }

  vector<pair<int, int>> build(vector<int> v) {
    auto cmp = [&](int a, int b) { return tin[a] < tin[b]; };
    sort(v.begin(), v.end(), cmp);
    for (int i = v.size() - 1; i > 0; --i) v.push_back(lca(v[i - 1],
      v[i]));
    sort(v.begin(), v.end(), cmp);
    v.erase(unique(v.begin(), v.end()), v.end());
    vector<pair<int, int>> res = {{v[0], -1}};
    for (int i = 1; i < v.size(); ++i) res.push_back({v[i], lca(v[i -
      1], v[i])});
    return res;
  }
};
```

math

berlekamp-massey

```cpp
// Berlekamp-Massey: finds the shortest linear recurrence of a
  sequence
// Time: O(N^2) where N is sequence size | Space: O(N)
// Note: requires at least 2L terms to correctly find a recurrence of
  length L
vector<int> berlekamp_massey(const vector<int> &seq) {
  vector<int> old = {1}, curr = {1};
  int prev = 1, len = 0, shift = 1;
  for (int i = 0; i < seq.size(); ++i) {
    int delta = 0;
    for (int j = 0; j <= len; ++j) {
      delta = (delta + curr[j] * seq[i - j]) % mod;
    }
    if (delta == 0) {
      shift++;
      continue;
    }
    vector<int> temp = curr;
    int mult = (mod - (delta * mod_pow(prev, mod - 2)) % mod) % mod;
    if (curr.size() <= old.size() + shift) {
      curr.resize(old.size() + shift + 1, 0);
    }
    for (int j = 0; j < old.size(); ++j) {
      curr[j + shift] = (curr[j + shift] + mult * old[j]) % mod;
    }
    if (2 * len <= i) {
      len = i + 1 - len;
      old = temp;
      prev = delta;
      shift = 1;
    } else {
      shift++;
    }
  }
  vector<int> coef(len);
  for (int i = 1; i <= len; ++i) {
    coef[i - 1] = (mod - curr[i]) % mod;
  }
  return coef;
}

// Linear Recurrence K-th Term: computes the K-th term of a sequence
// Time: O(L^2 log K) where L is recurrence length | Space: O(L)
// Note: 0-indexed | the sequence must have at least L terms
int kth_term(int k, const vector<int> &seq, const vector<int> &coef) {
  if (k < seq.size()) return seq[k];
  if (coef.empty()) return 0;

  int n = coef.size();

  auto combine = [&](const vector<int> &a, const vector<int> &b) ->
    vector<int> {
    vector<int> r(2 * n);
    for (int i = 0; i < n; ++i) {
      for (int j = 0; j < n; ++j) {
        r[i + j] = (r[i + j] + a[i] * b[j]) % mod;
      }
    }
    for (int i = 2 * n - 2; i >= n; --i) {
      for (int j = 0; j < n; ++j) {
        r[i - j - 1] = (r[i - j - 1] + r[i] * coef[j]) % mod;
      }
    }
    r.resize(n);
    return r;
  };

  vector<int> pol(n), exp(n);
  pol[0] = 1;
  if (n > 1) exp[1] = 1;
  else exp[0] = coef[0];

  while (k != 0) {
    if (k & 1) pol = combine(pol, exp);
    exp = combine(exp, exp);
    k >>= 1;
  }

  int ans = 0;
  for (int i = 0; i < n; ++i) {
    ans = (ans + pol[i] * seq[i]) % mod;
  }
  return ans;
}
```

fast-fourier-transform

convolution/

```cpp
vector<complex<float>> roots = {{0, 1}};

void fft(vector<complex<float>> &f, bool inv) {
  int n = f.size();
  for (int k = roots.size(); k < n; k *= 2) {
    roots.resize(2 * k);
    float ang = acosl(-1) / k;
    for (int j = 0; j < k; ++j) roots[k + j] = {cos(ang * j), sin(ang
      * j)};
  }
  vector<int> rev(n);
  for (int i = 1; i < n; ++i) {
    rev[i] = (rev[i / 2] + (i & 1) * n) / 2;
    if (i < rev[i]) swap(f[i], f[rev[i]]);
  }
  for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
      for (int j = 0; j < k; ++j) {
        complex<float> z = roots[k + j] * f[i + j + k];
        f[i + j + k] = f[i + j] - z;
        f[i + j] += z;
      }
    }
  }
```

```
  if (inv) {
    reverse(f.begin() + 1, f.end());
    for (complex<float> &x : f) x /= n;
  }
}

vector<int> convolution(const vector<int> &a, const vector<int> &b) {
  if (a.empty() or b.empty()) return {};
  int n = a.size() + b.size() - 1, m = 1;
  while (m < n) m *= 2;
  vector<complex<float>> f(m);
  for (int i = 0; i < a.size(); ++i) f[i].real(a[i]);
  for (int i = 0; i < b.size(); ++i) f[i].imag(b[i]);
  fft(f, false);
  for (complex<float> &x : f) x *= x;
  fft(f, true);
  vector<int> c(n);
  for (int i = 0; i < n; ++i) c[i] = round(f[i].imag() / 2);
  return c;
}
```

fast-walsh-hadamard-transform

convolution/

```
// Fast Walsh-Hadamard Transform: bitwise convolution, c[k] = sum of
  a[i] * b[j] over i opr j = k, with opr = '&', '|' or '^'
// Time: O(N log N) | Space: O(N)
void fwht(vector<int> &f, char opr, bool inv) {
  int n = f.size();
  for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
      for (int j = i; j < i + k; ++j) {
        int x = f[j], y = f[j + k];
        if (opr == '&') x += (inv) ? mod - y : y;
        if (opr == '|') y += (inv) ? mod - x : x;
        if (opr == '^') tie(x, y) = pair(x + y, x - y + mod);
        f[j] = x - (x >= mod) * mod, f[j + k] = y - (y >= mod) * mod;
      }
    }
  }
  if (opr == '^' and inv) {
    int inv_n = mod_pow(n, mod - 2, mod);
    for (int &x : f) x = x * inv_n % mod;
  }
}

vector<int> convolution(vector<int> a, vector<int> b, char opr) {
  int n = 1;
  while (n < max(a.size(), b.size())) n *= 2;
  a.resize(n), b.resize(n);
  fwht(a, opr, false), fwht(b, opr, false);
  for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % mod;
  fwht(a, opr, true);
  return a;
}
```

modulo-transform-technique

convolution/

```
vector<int> mtt(const vector<int> &a, const vector<int> &b, int mod) {
  const int p = 998244353, q = 167772161, r = 469762049;
  vector<int> x = convolution<p>(a, b), y = convolution<q>(a, b), z =
    convolution<r>(a, b);
  int p_inv = mod_pow(p, q - 2, q), pq_inv = mod_pow(p * q % r, r -
    2, r);
  for (int i = 0; i < x.size(); ++i) {
    int s = ((y[i] - x[i]) % q + q) % q * p_inv % q;
    int t = ((z[i] - x[i] - p * s) % r + r) % r * pq_inv % r;
    x[i] = (x[i] + p * s + p * q % mod * t) % mod;
  }
  return x;
}
```

number-theoretic-transform

convolution/

```
vector<int> roots = {0, 1};

template<int mod = 998244353> void ntt(vector<int> &f, bool inv) {
  static vector<int> roots = {0, 1};
  int n = f.size();
  for (int k = roots.size(); k < n; k *= 2) {
    roots.resize(2 * k);
    int w = mod_pow(3, (mod - 1) / (2 * k), mod);
    for (int j = 0; j < k; ++j) roots[k + j] = (j == 0) ? 1 : roots[k
      + j - 1] * w % mod;
  }
  vector<int> rev(n);
  for (int i = 1; i < n; ++i) {
    rev[i] = (rev[i / 2] + (i & 1) * n) / 2;
    if (i < rev[i]) swap(f[i], f[rev[i]]);
  }
  for (int k = 1; k < n; k *= 2) {
    for (int i = 0; i < n; i += 2 * k) {
      for (int j = 0; j < k; ++j) {
        int x = f[i + j], z = roots[k + j] * f[i + j + k] % mod;
        f[i + j] = (x + z < mod) ? x + z : x + z - mod;
        f[i + j + k] = (x - z >= 0) ? x - z : x - z + mod;
      }
    }
  }
  if (inv) {
    reverse(f.begin() + 1, f.end());
    int inv_n = mod_pow(n, mod - 2, mod);
    for (int &x : f) x = x * inv_n % mod;
  }
}

template<int mod = 998244353> vector<int> convolution(vector<int> a,
  vector<int> b) {
  if (a.empty() or b.empty()) return {};
  int n = a.size() + b.size() - 1, m = 1;
  while (m < n) m *= 2;
  a.resize(m), b.resize(m);
  for (int i = 0; i < m; ++i) a[i] %= mod, b[i] %= mod;
  ntt<mod>(a, false), ntt<mod>(b, false);
  for (int i = 0; i < m; ++i) a[i] = a[i] * b[i] % mod;
  ntt<mod>(a, true);
  a.resize(n);
  return a;
}
```

gaussian-elimination

gaussian-elimination/

```
// Gaussian Elimination: solves A x = b from the augmented matrix a =
  [A | b], with n equations and m variables
// Time: O(n m min(n, m)) | Space: O(m) auxiliary
// Note: uses mod and mod_pow | a is int (values in [0, mod)) or
  float | returns {0: none, 1: unique, 2: infinite, x}, free variables
  are 0
template<class T> pair<int, vector<T>>
  gaussian_elimination(vector<vector<T>> &a) {
  constexpr bool real = is_floating_point_v<T>;
  auto fix = [](T x) -> T { if constexpr (real) return x; else return
    (x % mod + mod) % mod; };
  auto inv = [](T x) -> T { if constexpr (real) return 1 / x; else
    return mod_pow(x, mod - 2, mod); };
  auto zero = [](T x) -> bool { if constexpr (real) return abs(x) <
    1e-9; else return x == 0; };

  int n = a.size(), m = a[0].size() - 1, r = 0;
  vector<int> col(n);
  for (int c = 0; c < m and r < n; ++c) {
    int k = r;
    for (int i = r; i < n; ++i) if (abs(a[i][c]) > abs(a[k][c])) k =
      i;
    if (zero(a[k][c])) continue;
    swap(a[r], a[k]);
    T d = inv(a[r][c]);
    for (int j = c; j <= m; ++j) a[r][j] = fix(a[r][j] * d);
    for (int i = 0; i < n; ++i) {
      if (i == r or zero(a[i][c])) continue;
      T f = a[i][c];
      for (int j = c; j <= m; ++j) a[i][j] = fix(a[i][j] - f *
        a[r][j]);
    }
    col[r++] = c;
  }
  for (int i = r; i < n; ++i) if (not zero(a[i][m])) return {0, {}};
  vector<T> x(m);
  for (int i = 0; i < r; ++i) x[col[i]] = a[i][m];
  return {r == m ? 1 : 2, x};
}
```

xor-gaussian-elimination

gaussian-elimination/

```
// XOR Gaussian Elimination: solves XOR systems (mod 2)
// Time: O(N * n * m / 64) | Space: O(N * m / 64) auxiliary
// Note: returns {(-1: none, r: rank), solution} | 2^(n - r)
  solutions and undetermined vars = d
template<int N> pair<int, vector<int>> xor_gaussian_elimination(int
  n, vector<bitset<N + 1>> a, int d) {
  int m = a.size(), r = 0;
  vector<int> where(n, -1);
  for (int c = 0; c < n and r < m; ++c) {
    int p = r;
    while (p < m and not a[p][c]) {
      p++;
    }
    if (p == m) continue;
    swap(a[r], a[p]);
    for (int i = 0; i < m; ++i) {
      if (i != r and a[i][c]) a[i] ^= a[r];
    }
    where[c] = r++;
  }
  for (int i = r; i < m; ++i) {
    if (a[i][n]) return {-1, {}};
  }
  vector<int> x(n, d);
  bitset<N + 1> ones;
  for (int j = 0; j < n; ++j) {
    if (where[j] == -1) ones[j] = d;
  }
  for (int j = 0; j < n; ++j) {
    if (where[j] != -1) x[j] = a[where[j]][n] ^ ((a[where[j]] &
      ones).count() & 1);
  }
  return {r, x};
}
```

matrix

```
// Matrix: product and fast power mod | counts walks of length k and
  linear recurrences
// Time: product O(N^3), power O(N^3 log K) | Space: O(N^2)
// Min-Plus: shortest walk with k edges | fill with inf and identity
  diagonal 0, r[i][j] = min(r[i][j], a[i][k] + b[k][j])
// Max-Plus: longest walk with k edges | fill with -inf and identity
  diagonal 0, r[i][j] = max(r[i][j], a[i][k] + b[k][j])
using matrix = vector<vector<int>>;

matrix operator*(const matrix &a, const matrix &b) {
  int n = a.size();
  int m = b[0].size();
  int h = b.size();
  matrix r(n, vector<int>(m));
  for (int i = 0; i < n; ++i) {
    for (int k = 0; k < h; ++k) {
      for (int j = 0; j < m; ++j) {
        r[i][j] = (r[i][j] + a[i][k] * b[k][j]) % mod;
      }
    }
  }
  return r;
}

matrix mod_pow(matrix a, int e) {
  int n = a.size();
  matrix r(n, vector<int>(n));
  for (int i = 0; i < n; ++i) {
    r[i][i] = 1;
  }
  while (e != 0) {
    if (e & 1) r = r * a;
    a = a * a;
    e >>= 1;
  }
  return r;
}
```

nim

```
vector<int> grundy(int n, vector<int> &moves) {
  vector<int> g(n + 1), seen(moves.size() + 1, -1);
  for (int i = 1; i <= n; ++i) {
    for (int m : moves) if (m <= i) seen[g[i - m]] = i;
    while (seen[g[i]] == i) g[i]++;
  }
  return g;
}

bool nim(vector<int> &piles, vector<int> &moves) {
  vector<int> g = grundy(*max_element(piles.begin(), piles.end()),
    moves);
  int x = 0;
  for (int p : piles) x ^= g[p];
  return x != 0;
}

bool misere(vector<int> &piles) {
  int x = 0;
  for (int p : piles) x ^= p;
  if (*max_element(piles.begin(), piles.end()) > 1) return x != 0;
  return x == 0;
}
```

xor-basis

```
// XOR Basis: reduced basis of the inserted numbers, answering
  questions about the xors of their subsets
// Time: O(bits) per operation | Space: O(bits)
// Note: basis[i] has highest bit i and no other pivot bit | kth(k) =
  k-th smallest distinct subset xor, from 0 | [k] counts all 2^n
  subsets, with repeats | max(x) = largest x ^ subset xor
struct xor_basis {
  static const int bits = 60;
  int n = 0, r = 0;
  array<int, bits> basis = {};

  bool insert(int x) {
    n++;
    for (int i = bits - 1; i >= 0; --i) {
      if (x >> i & 1 and basis[i] != 0) x ^= basis[i];
    }
    if (x == 0) return false;
```

```
    int p = __lg(x);
    for (int i = p + 1; i < bits; ++i) {
      if (basis[i] >> p & 1) basis[i] ^= x;
    }
    basis[p] = x, r++;
    return true;
  }

  bool contains(int x) {
    for (int i = bits - 1; i >= 0; --i) {
      if (x >> i & 1) x ^= basis[i];
    }
    return x == 0;
  }

  int max(int x = 0) {
    for (int i = bits - 1; i >= 0; --i) {
      if (not (x >> i & 1)) x ^= basis[i];
    }
    return x;
  }

  int kth(int k) {
    if (k >> r != 0) return -1;
    int x = 0;
    for (int i = 0; i < bits; ++i) {
      if (basis[i] == 0) continue;
      if (k & 1) x ^= basis[i];
      k >>= 1;
    }
    return x;
  }

  int operator[](int k) {
    return n - r >= 63 ? 0 : kth(k >> (n - r));
  }

  int ways(int x) {
    return contains(x) ? mod_pow(2, n - r, mod) : 0;
  }
};

// XOR Basis: reduced basis of the inserted bits-bit numbers,
  answering questions about the xors of their subsets
// Time: O(bits^2 / 64) per operation | Space: O(bits^2 / 64)
// Note: basis[i] has highest bit i and no other pivot bit | kth(k) =
  k-th smallest distinct subset xor, from 0, needs k < 2^r | [k]
  counts all 2^n subsets, with repeats | max(x) = largest x ^ subset
  xor
template<int bits>
struct xor_basis {
  int n = 0, r = 0;
  vector<bitset<bits>> basis = vector<bitset<bits>>(bits);

  bool insert(bitset<bits> x) {
    n++;
    for (int i = bits - 1; i >= 0; --i) {
      if (x[i] and basis[i][i]) x ^= basis[i];
    }
    if (x.none()) return false;
    int p = bits - 1;
    while (not x[p]) p--;
    for (int i = p + 1; i < bits; ++i) {
      if (basis[i][p]) basis[i] ^= x;
    }
    basis[p] = x, r++;
    return true;
  }

  bool contains(bitset<bits> x) {
    for (int i = bits - 1; i >= 0; --i) {
      if (x[i] and basis[i][i]) x ^= basis[i];
    }
    return x.none();
  }

  bitset<bits> max(bitset<bits> x = {}) {
    for (int i = bits - 1; i >= 0; --i) {
      if (not x[i] and basis[i][i]) x ^= basis[i];
    }
    return x;
  }

  bitset<bits> kth(bitset<bits> k) {
    bitset<bits> x;
    for (int i = 0, j = 0; i < bits; ++i) {
      if (basis[i][i] and k[j++]) x ^= basis[i];
    }
    return x;
  }

  bitset<bits> operator[](int k) {
    return kth(n - r >= 63 ? 0 : k >> (n - r));
  }

  int ways(bitset<bits> x) {
    return contains(x) ? mod_pow(2, n - r, mod) : 0;
  }
};
```

number-theory

floor-ceil

```
int floor(int x, int y) {
  return (x / y) - (((x ^ y) < 0) & x % y != 0);
}

int ceil(int x, int y) {
  return (x / y) + (((x ^ y) >= 0) & x % y != 0);
}
```

floor-sum

```
// Floor Sum: calculates sum_{i=0}^{n-1} floor((a * i + b) / c)
// Time: O(log c) | Space: O(log c)
// Note: requires n >= 0 and c >= 1
int floor_sum(int n, int a, int b, int c) {
  if (n == 0) return 0;
  int x = (a % c + c) % c, y = (b % c + c) % c;
  int r = n * (n - 1) / 2 * ((a - x) / c) + n * ((b - y) / c);
  int t = x * n + y;
  return r + floor_sum(t / c, c, t % c, x);
}

// Floor Sum Range: calculates sum_{i=l}^{r-1} floor((a * i + b) / c)
// Time: O(log c) | Space: O(log c)
// Note: requires l < r and c >= 1
int floor_sum_range(int l, int r, int a, int b, int c) {
  return floor_sum(r - l, a, a * l + b, c);
}
```

harmonic-lemma

```
// Harmonic Lemma: quotient blocks in a specific range [l, r]
// Time: O(sqrt(N)) | Space: O(1)
// Note: iterates over ranges [i, j] inside [l, r] where floor(n / k)
  is exactly q
int harmonic_lemma(int n, int l, int r) {
  int s = 0;
  for (int i = l, j; i <= r; i = j + 1) {
    int q = n / i;
    if (q == 0) {
      j = r;
    } else {
      j = min(r, n / q);
    }
    s += q * (j - i + 1);
  }
  return s;
}
```

mobius-inversion

Inversão de Möbius

Problema base. Inverter somas sobre divisores e contar pares com condição de gcd (coprimos, $\gcd = x$).

Fórmula geral.

$$\boxed{g(n) = \sum_{d \mid n} f(d) \iff f(n) = \sum_{d \mid n} \mu(d)\, g\left(\frac{n}{d}\right)}$$

onde μ é a função de Möbius:

- $\mu(1) = 1$;
- $\mu(d) = 0$ se algum primo aparece ao quadrado em d ($p^2 \mid d$);
- $\mu(d) = (-1)^k$ se d é produto de k primos distintos.

Propriedade fundamental. Com $[P] = 1$ se P é verdadeira e 0 se não, ela troca a condição "coprimos" por uma soma sobre os divisores comuns:

$$[\gcd(a, b) = 1] = \sum_{d \mid \gcd(a,b)} \mu(d) = \sum_{d \mid a,\ d \mid b} \mu(d)$$

Roteiro. Escreva a contagem como uma soma de $[\gcd = 1]$, troque pela soma de $\mu(d)$ e inverta a ordem das somas: para cada d, só sobra contar quantos objetos têm tudo múltiplo de d.

Pares com $\gcd = x$ num vetor

Seja $C(v)$ a quantidade de elementos múltiplos de v (crivo em $O(M \log M)$, com M o maior valor). $\gcd(A_i, A_j) = x$ quando os dois são múltiplos de x e, divididos por x, ficam coprimos:

$$\text{Pares} = \sum_{\substack{i,j \\ x \mid A_i,\ x \mid A_j}} \left[\gcd\left(\tfrac{A_i}{x}, \tfrac{A_j}{x}\right) = 1\right].$$

Pela propriedade, com k dividindo $\frac{A_i}{x}$ e $\frac{A_j}{x}$, ou seja, kx dividindo A_i e A_j:

$$\text{Pares} = \sum_{\substack{i,j \\ x \mid A_i,\ x \mid A_j}} \ \sum_{kx \mid A_i,\ kx \mid A_j} \mu(k).$$

Invertendo a ordem, cada k soma $\mu(k)$ uma vez para cada par com os dois múltiplos de kx, e há $C(kx)^2$ desses pares:

$$\text{Pares} = \sum_{k=1}^{\lfloor M/x \rfloor} \mu(k)\, C(kx)^2 \qquad (O(M/x) \text{ por consulta}).$$

São pares ordenados, contando $i = j$; para pares $i < j$, troque C^2 por $\binom{C}{2}$.

Pares em $[1, n] \times [1, m]$ com $\gcd = x$

Os pares com $\gcd = x$ são (xi', xj') com i', j' coprimos, $i' \le n/x$ e $j' \le m/x$:

$$\text{Pares} = \sum_{i \le n,\ j \le m} [\gcd(i, j) = x] = \sum_{i' \le n/x} \ \sum_{j' \le m/x} [\gcd(i', j') = 1].$$

Pela propriedade e invertendo a ordem, cada d conta os pares com $d \mid i'$ e $d \mid j'$: há $\lfloor \frac{n}{xd} \rfloor$ múltiplos de d até $\frac{n}{x}$ e $\lfloor \frac{m}{xd} \rfloor$ até $\frac{m}{x}$:

$$\text{Pares} = \sum_{d \ge 1} \mu(d) \left\lfloor \tfrac{n}{xd} \right\rfloor \left\lfloor \tfrac{m}{xd} \right\rfloor.$$

Com $x = 1$ são os pares coprimos. Roda em $O(\sqrt{n} + \sqrt{m})$ agrupando os d com os mesmos quocientes e usando prefix sum de μ.

Subconjuntos com $\gcd = 1$ num vetor

Com o mesmo C e somando sobre os subconjuntos S não vazios do vetor:

$$\text{Subconjuntos} = \sum_{S} [\gcd(S) = 1] = \sum_{S} \ \sum_{d \mid \gcd(S)} \mu(d).$$

$d \mid \gcd(S)$ quando d divide todos os elementos de S. Invertendo a ordem, cada d conta os subconjuntos não vazios formados só por múltiplos de d, que são $2^{C(d)} - 1$:

$$\text{Subconjuntos} = \sum_{d=1}^{M} \mu(d) \left(2^{C(d)} - 1\right).$$

mobius

```
int mobius(int n) {
  int mu = 1;
  for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
    n /= i;
    if (n % i == 0) return 0;
    mu *= -1;
  }
  if (n > 1) mu *= -1;
  return mu;
}

vector<int> mobius_sieve(int n) {
  vector<int> mu(n + 1);
  vector<bool> is_prime(n + 1, true);
  vector<int> primes;
  mu[1] = 1;
  is_prime[1] = false;
  for (int i = 2; i <= n; ++i) {
    if (is_prime[i]) {
      primes.push_back(i);
      mu[i] = -1;
    }
    for (int p : primes) {
```

```
      if (i * p > n) break;
      is_prime[i * p] = false;
      if (i % p == 0) {
        mu[i * p] = 0;
        break;
      } else {
        mu[i * p] = -mu[i];
      }
    }
  }
  return mu;
}
```

chinese-remainder-theorem

mod/

```
int128 chinese_remainder_theorem(const vector<int> &rem, const
  vector<int> &mod) {
  int128 r = rem[0], m = mod[0];
  for (int i = 1; i < rem.size(); ++i) {
    int g = gcd(static_cast<int>(m % mod[i]), mod[i]), h = mod[i] / g;
    if ((rem[i] - r) % g != 0) return -1;

    auto [x, _] = extended_gcd(static_cast<int>(m / g % h), h);

    r += m * (((((rem[i] - r) / g) * x) % h + h) % h);
    m *= h;
  }
  return r;
}
```

exponent-reduction

mod/

Redução de Expoentes (Teorema de Euler Estendido)

Problema base. Calcular $a^e \bmod m$ quando o expoente e é enorme (string com 10^5 dígitos, torre a^{b^c}).

Ideia. As potências de a módulo m entram num ciclo de tamanho que divide $\varphi(m)$, e o ciclo começa até o expoente $\varphi(m)$. Vale para **quaisquer** a e m, mesmo com $\gcd(a, m) > 1$:

$$a^e \equiv \begin{cases} a^e, & \text{se } e < \varphi(m) \\ a^{(e \bmod \varphi(m)) + \varphi(m)}, & \text{se } e \geq \varphi(m) \end{cases} \pmod m$$

- **Caso $e < \varphi(m)$:** não reduza o expoente; reduzir antes do ciclo muda o resultado quando $\gcd(a, m) > 1$.
- **Caso $e \geq \varphi(m)$:** troque e por $(e \bmod \varphi(m)) + \varphi(m)$; o $+\varphi(m)$ mantém o expoente dentro do ciclo.
- **Se $\gcd(a, m) = 1$ (Euler):** o ciclo começa em 0, então basta $a^{e \bmod \varphi(m)}$.

extended-gcd

mod/

```
// Extended GCD: finds coefficients x and y such that ax + by =
  gcd(a, b)
// Time: O(log(min(a, b))) | Space: O(log(min(a, b)))
// Note: returns [x0, y0] | all valid pairs are: x = x0 + k*(b/gcd)
  and y = y0 - k*(a/gcd)
pair<int, int> extended_gcd(int a, int b) {
  if (b == 0) return {1, 0};

  auto [x, y] = extended_gcd(b, a % b);
  return {y, x - (a / b) * y};
}
```

mod-log

mod/

```
int mod_log(int a, int b, int mod) {
  a %= mod, b %= mod;
  int k = 1, steps = 0;
  for (int g = gcd(a, mod); g > 1; g = gcd(a, mod)) {
    if (b == k) return steps;
    if (b % g != 0) return -1;
    b /= g, mod /= g, steps++;
    k = k * (a / g) % mod;
  }
  int n = sqrt(mod) + 1, giant = 1;
  for (int i = 0; i < n; ++i) giant = giant * a % mod;
  vector<array<int, 2>> baby;
  for (int q = 0, cur = b; q <= n; ++q) {
    baby.push_back({cur, -q});
    cur = cur * a % mod;
  }
  sort(baby.begin(), baby.end());
  for (int p = 1, cur = k; p <= n; ++p) {
    cur = cur * giant % mod;
    auto itr = lower_bound(baby.begin(), baby.end(), array<int,
      2>{cur, -n});
    if (itr != baby.end() and (*itr)[0] == cur) return n * p +
      (*itr)[1] + steps;
  }
  return -1;
}
```

multi-extended-gcd

mod/

```
vector<int> multi_extended_gcd(vector<int> &a) {
  int n = a.size(), g = a[0];
  vector<int> c(n, 1), h(n - 1);

  for (int i = 1; i < n; ++i) {
    auto [x, y] = extended_gcd(g, a[i]);
    g = gcd(g, a[i]);
    h[i - 1] = x;
    c[i] = y;
  }

  int p = 1;
  for (int i = n - 1; i > 0; --i) {
    c[i] *= p;
    p *= h[i - 1];
  }
  c[0] *= p;
  return c;
}
```

linear-sieve

prime/

```
vector<int> linear_sieve(int n) {
  vector<int> p, spf(n + 1);
  for (int i = 2; i <= n; ++i) {
    if (spf[i] == 0) {
      spf[i] = i;
      p.push_back(i);
    }
    for (int j : p) {
      if (j > spf[i] or i * j > n) break;
      spf[i * j] = j;
    }
  }
  return p;
}
```

miller-rabin

prime/

```
bool miller_rabin(int n) {
  if (n < 2) return false;
  for (int p : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
    if (n % p == 0) return n == p;
  }
  int s = __builtin_ctzll(n - 1), d = (n - 1) >> s;
  for (int a : {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
    if (a % n == 0) continue;
    int128 x = 1, b = a % n;
    for (int e = d; e > 0; e /= 2) {
      if (e & 1) x = x * b % n;
      b = b * b % n
    }
    if (x == 1) continue;
    for (int i = 1; i < s and x != n - 1; ++i) x = x * x % n;
    if (x != n - 1) return false;
  }
  return true;
}
```

pollard-rho

prime/

```
int pollard_rho(int n) {
  if (n % 2 == 0) return 2;
  for (int c = 1;; ++c) {
    auto f = [&](int x) { return (int) (((int128) x * x + c) % n); };
    int x = 2, y = 2, q = 1, g = 1;
    for (int r = 1; g == 1; r *= 2) {
      x = y;
      for (int i = 0; i < r and g == 1; ++i) {
        y = f(y);
        q = (int128) q * abs(x - y) % n;
        if (i % 128 == 127 or i == r - 1) g = gcd(q, n);
      }
    }
    if (g != n) return g;
  }
}

vector<int> factorize(int n) {
  vector<int> primes, todo = {n};
  while (todo.size()) {
    int m = todo.back(); todo.pop_back();
    if (m == 1) continue;
    if (miller_rabin(m)) {
      primes.push_back(m);
    } else {
      int d = pollard_rho(m);
      todo.push_back(d), todo.push_back(m / d);
    }
  }
  sort(primes.begin(), primes.end());
  return primes;
}
```

segmented-sieve

prime/

```
// Segmented Sieve: primes in [l, r], crossing out the multiples of
  the primes up to sqrt(r)
// Time: O((r - l) log log r + sqrt(r)) | Space: O(r - l + sqrt(r))
// Note: uses linear_sieve | is_prime[i - l] tells if i is prime
vector<int> segmented_sieve(int l, int r) {
  vector<int> primes = linear_sieve(sqrt(r));
  vector<bool> is_prime(r - l + 1, true);
  for (int p : primes) {
    for (int i = max(p * p, (l + p - 1) / p * p); i <= r; i += p) {
      is_prime[i - l] = false;
    }
  }
  primes.clear();
  for (int i = max<int>(2, l); i <= r; ++i) {
    if (is_prime[i - l]) primes.push_back(i);
  }
  return primes;
}
```

totient

```
int totient(int n) {
  int phi = n;
  for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
    phi -= phi / i;
    while (n % i == 0) n /= i;
  }
  if (n > 1) phi -= phi / n;
  return phi;
}

vector<int> totient_sieve(int n) {
  vector<int> phi(n + 1);
  for (int i = 1; i <= n; ++i) {
    phi[i] = i;
  }
  for (int i = 2; i <= n; ++i) if (phi[i] == i) {
    for (int j = i; j <= n; j += i) {
      phi[j] -= phi[j] / i;
    }
  }
  return phi;
}
```

square-root

mo-3d

```
// Mo's Algorithm 3D: offline range queries with point updates
// Time: O(N^(5/3)) | Space: O(N + Q + U)
// Q elements: {l, r, t, id} | t = # of updates prior to query | id =
  original query index
// U elements: {i, x} | i = array index to update | x = new value
// M parameter: block size multiplier to tune the N^(2/3) block size
// Note: implement 'insert', 'remove', and 'curr' state according to
  your specific problem
vector<int> mo_3d(vector<int> &a, vector<array<int, 4>> &q,
  vector<array<int, 2>> &u, float m = 1.0) {
  int b = clamp<int>(m * pow(a.size(), 2.0 / 3.0), 1, a.size());
  auto block = [&](int i) -> int {
    return i / b;
  };

  sort(q.begin(), q.end(), [&](const array<int, 4> &x, const
    array<int, 4> &y) {
    if (block(x[0]) != block(y[0])) return block(x[0]) < block(y[0]);
    if (block(x[1]) != block(y[1])) return (block(x[0]) & 1) ?
      block(x[1]) < block(y[1]) : block(x[1]) > block(y[1]);
    return (block(x[1]) & 1) ? x[2] < y[2] : x[2] > y[2];
  });

  int curr = 0;
  auto insert = [&](int x) -> void {};
  auto remove = [&](int x) -> void {};

  auto update = [&](int t, int l, int r) -> void {
    int i = u[t][0];
    if (l <= i and i <= r) {
      remove(a[i]);
      swap(a[i], u[t][1]);
      insert(a[i]);
    } else {
      swap(a[i], u[t][1]);
    }
  };

  vector<int> ans(q.size());
  int h = 0, i = 0, j = -1;
  for (auto &[l, r, t, id] : q) {
    while (h < t) update(h++, i, j);
```

```
    while (h > t) update(--h, i, j);
    while (i > l) insert(a[--i]);
    while (j < r) insert(a[++j]);
    while (i < l) remove(a[i++]);
    while (j > r) remove(a[j--]);
    ans[id] = curr;
  }
  return ans;
}
```

mo-tree

```
// Mo's Algorithm on Trees: offline path queries on vertex values
// Time: O((N + Q) sqrt N) | Space: O(N + Q)
// Note: requires hilbert (mo) | q elements: {u, v, lca(u, v)} |
  implement 'insert', 'remove' and 'curr' like in mo
vector<int> mo_tree(vector<vector<int>> &g, vector<int> &a,
  vector<array<int, 3>> &q) {
  int n = g.size();
  vector<int> tin(n), tout(n), order;
  auto dfs = [&](auto &self, int u, int p) -> void {
    tin[u] = order.size();
    order.push_back(u);
    for (int v : g[u]) if (v != p) self(self, v, u);
    tout[u] = order.size();
    order.push_back(u);
  };
  dfs(dfs, 0, -1);

  vector<array<int, 5>> qs;
  for (int k = 0; k < q.size(); ++k) {
    auto [u, v, p] = q[k];
    if (tin[u] > tin[v]) swap(u, v);
    int l = p == u ? tin[u] : tout[u], r = tin[v];
    qs.push_back({hilbert(l, r), l, r, p == u ? -1 : p, k});
  }
  sort(qs.begin(), qs.end());

  int curr = 0;
  auto insert = [&](int x) -> void {};
  auto remove = [&](int x) -> void {};

  vector<bool> on(n);
  auto toggle = [&](int u) -> void {
    if (on[u]) remove(a[u]);
    else insert(a[u]);
    on[u] = not on[u];
  };

  vector<int> ans(q.size());
  int i = 0, j = -1;
  for (auto [h, l, r, p, id] : qs) {
    while (i > l) toggle(order[--i]);
    while (j < r) toggle(order[++j]);
    while (i < l) toggle(order[i++]);
    while (j > r) toggle(order[j--]);
    if (p != -1) insert(a[p]);
    ans[id] = curr;
    if (p != -1) remove(a[p]);
  }
  return ans;
}
```

mo

```
int hilbert(int x, int y) {
  int h = 0;
  for (int s = 25; s >= 0; --s) {
    int rx = (x >> s) & 1;
    int ry = (y >> s) & 1;
    int qd = (3 * rx) ^ ry;
    h = (h << 2) | qd;
    if (ry == 0) {
      x ^= -rx;
      y ^= -rx;
      swap(x, y);
    }
  }
  return h;
}

vector<int> mo(vector<int> &a, vector<array<int, 4>> &q) {
  for (auto &[h, l, r, id] : q) h = hilbert(l, r);
  sort(q.begin(), q.end());

  int curr = 0;
  auto insert = [&](int x) -> void {};
  auto remove = [&](int x) -> void {};

  vector<int> ans(q.size());
  int i = 0, j = -1;
  for (auto &[h, l, r, id] : q) {
    while (i > l) insert(a[--i]);
    while (j < r) insert(a[++j]);
    while (i < l) remove(a[i++]);
    while (j > r) remove(a[j--]);
    ans[id] = curr;
  }
  return ans;
}
```

sqrt-decomposition

```
// Sqrt Decomposition: point set, range add and range sum with blocks
  of size m = sqrt(n)
// Time: O(n / m + m) per operation | Space: O(n)
// Note: 0-based | the value at i is arr[i] + lazy[i / m]
struct sqrt_decomposition {
  int n, m;
  vector<int> arr, sum, lazy;

  void build(int b) {
    sum[b] = 0;
    for (int i = first(b); i <= last(b); ++i) {
      sum[b] += arr[i];
    }
  }

  sqrt_decomposition(const vector<int> &arr): n(arr.size()), arr(arr)
    {
    m = max<int>(1, sqrt(n));
    sum.assign(n / m + 1, 0);
    lazy.assign(n / m + 1, 0);
    for (int b = 0; b <= n / m; ++b) build(b);
  }

  int first(int b) {
    return b * m;
  }

  int last(int b) {
    return min(n, b * m + m) - 1;
  }

  void set(int i, int x) {
    int b = i / m;
    arr[i] = x - lazy[b];
    build(b);
  }

  void update(int l, int r, int x) {
    int bl = l / m, br = r / m;
    if (bl >= br) {
      for (int i = l; i <= r; ++i) arr[i] += x;
      build(bl);
      return;
    }
    for (int b = bl + 1; b < br; ++b) lazy[b] += x;
    update(l, last(bl), x);
    update(first(br), r, x);
  }

  int query(int l, int r) {
    int bl = l / m, br = r / m, ans = 0;
    if (bl >= br) {
      for (int i = l; i <= r; ++i) ans += arr[i] + lazy[bl];
      return ans;
    }
    for (int b = bl + 1; b < br; ++b) ans += sum[b] + lazy[b] * m;
    return ans + query(l, last(bl)) + query(first(br), r);
  }
};
```

string

aho-corasick

```
struct aho_corasick {
  struct node {
    int len = 0, end = 0;
    int fail = 0, exit = 0;
    array<int, 26> next = {};
  };

  vector<node> tree = {node()};
  vector<int> order;

  int insert(const string &s) {
    int a = 0;
    for (char h : s) {
      int c = h - 'a';
      if (tree[a].next[c] == 0) {
        tree[a].next[c] = tree.size();
        tree.emplace_back();
      }
      a = tree[a].next[c];
    }
    tree[a].end++;
    return a;
  }

  void build() {
    order = {0};
    for (int i = 0; i < order.size(); ++i) {
      int a = order[i], f = tree[a].fail;
      tree[a].exit = (tree[f].end > 0) ? f : tree[f].exit;
      for (int c = 0; c < 26; ++c) {
        int b = tree[a].next[c];
        if (b == 0) {
          tree[a].next[c] = tree[f].next[c];
        } else {
          tree[b].fail = (a == 0) ? 0 : tree[f].next[c];
          tree[b].len = tree[a].len + 1;
          order.push_back(b);
        }
      }
    }
  }

  vector<array<int, 2>> match(const string &t) {
    vector<array<int, 2>> p;
    int a = 0;
    for (int i = 0; i < t.size(); ++i) {
      a = tree[a].next[t[i] - 'a'];
      for (int b = a; b != 0; b = tree[b].exit) {
        if (tree[b].end > 0) p.push_back({i - tree[b].len + 1, b});
      }
    }
    return p;
  }
};
```

dynamic-hasher

hash/

```
info combine(const info &a, const info &b) {
  info c;
  c.len = a.len + b.len;
  c.norm = (a.norm * b.pow + b.norm) % mod;
  c.rev = (b.rev * a.pow + a.rev) % mod;
  c.pow = a.pow * b.pow % mod;
  return c;
}

struct dynamic_hasher {
  int n;
  vector<info> tree;

  dynamic_hasher(const string &s): n(s.size()), tree(2 * n) {
    for (int i = 0; i < n; ++i) tree[n + i] = {1, s[i], s[i], base};
    for (int i = n - 1; i > 0; --i) tree[i] = combine(tree[2 * i],
      tree[2 * i + 1]);
  }

  void update(int i, char c) {
    tree[i += n] = {1, c, c, base};
    for (i /= 2; i > 0; i /= 2) tree[i] = combine(tree[2 * i], tree[2
      * i + 1]);
  }

  info query(int l, int r) {
    info a, b;
    for (l += n, r += n + 1; l < r; l /= 2, r /= 2) {
      if (l & 1) a = combine(a, tree[l++]);
      if (r & 1) b = combine(tree[--r], b);
    }
    return combine(a, b);
  }
};
```

hasher

hash/

```
const int mod = (1LL << 61) - 1;
const int128 base = uniform_int_distribution<int>(256, mod - 1)(rng);

struct info {
  int len = 0, norm = 0, rev = 0;
  int128 pow = 1;

  bool operator==(const info &o) const {
    return len == o.len and norm == o.norm;
  }
};

struct hasher {
  int n;
  vector<int> pre, suf;
  vector<int128> pows;

  hasher(const string &s): n(s.size()), pre(n + 1), suf(n + 1),
    pows(n + 1, 1) {
    for (int i = 0; i < n; ++i) {
```

```
      pre[i + 1] = (pre[i] * base + s[i]) % mod;
      pows[i + 1] = pows[i] * base % mod;
    }
    for (int i = n - 1; i >= 0; --i) suf[i] = (suf[i + 1] * base +
      s[i]) % mod;
  }

  info query(int l, int r) {
    info h;
    h.len = r - l + 1;
    h.pow = pows[h.len];
    h.norm = (pre[r + 1] - pre[l] * h.pow % mod + mod) % mod;
    h.rev = (suf[l] - suf[r + 1] * h.pow % mod + mod) % mod;
    return h;
  }
};
```

longest-prefix-suffix

```
vector<int> longest_prefix_suffix(string &s) {
  int n = s.length();
  vector<int> lps(n);
  for (int i = 1; i < n; ++i) {
    int j = lps[i - 1];
    while (j != 0 and s[i] != s[j]) j = lps[j - 1];
    if (s[i] == s[j]) j++;
    lps[i] = j;
  }
  return lps;
}
```

palindromic-tree

```
struct palindromic_tree {
  struct node {
    int length, fail;
    array<int, 26> child = {};

    node(int length): length(length), fail(0) {}
  };

  int suffix;
  vector<char> text;
  vector<node> tree;

  palindromic_tree(int n): suffix(0) {
    text.reserve(n + 5);
    text.push_back('$');
    tree.reserve(n + 5);
    tree.push_back(node(0));
    tree.push_back(node(-1));
    tree[0].fail = 1;
  }

  int follow(int id) {
    while (text[(text.size() - tree[id].length) - 2] != text.back()) {
      id = tree[id].fail;
    }
    return id;
  }

  void insert(char c) {
    text.push_back(c);
    suffix = follow(suffix);

    if (not tree[suffix].child[c - 'a']) {
      int id = tree.size();
      tree.push_back(node(tree[suffix].length + 2));

      tree[id].fail = tree[follow(tree[suffix].fail)].child[c - 'a'];
      tree[suffix].child[c - 'a'] = id;
    }
    suffix = tree[suffix].child[c - 'a'];
  }
};
```

prefix-automaton

```
vector<array<int, 26>> prefix_automaton(const string &s) {
  int n = s.size();
  vector<array<int, 26>> aut(n + 1);
  if (n > 0) aut[0][s[0] - 'a'] = 1;
  for (int i = 1, j = 0; i <= n; ++i) {
    aut[i] = aut[j];
    if (i < n) {
      aut[i][s[i] - 'a'] = i + 1;
      j = aut[j][s[i] - 'a'];
    }
  }
  return aut;
}
```

suffix-array

suffix-array/

```
// Suffix Array: sorts all suffixes of s, with pattern search and
  k-th smallest substring (counting repeats or not)
// Time: build O(N log N) | search O(M log N) | kth O(N log N) |
  kth_distinct O(N) | Space: O(N)
// Note: suf = sorted suffix starts | rank = position of the suffix i
  in suf | search = [l, r) | kth = (start, len)
struct suffix_array {
  int n;
  string s;
  vector<int> suf, rank, lcp;

  suffix_array(const string &s): n(s.size()), s(s), suf(n + 1),
    rank(n + 1) {
    vector<int> temp(n + 1), cnt(max<int>(256, n + 1));
    auto sort = [&]() {
      fill(cnt.begin(), cnt.end(), 0);
      for (int i = 0; i <= n; ++i) cnt[rank[i]]++;
      for (int i = 1; i < cnt.size(); ++i) cnt[i] += cnt[i - 1];
      for (int i = n; i >= 0; --i) temp[--cnt[rank[suf[i]]]] = suf[i];
      swap(suf, temp);
    };
    for (int i = 0; i < n; ++i) rank[i] = s[i];
    iota(suf.begin(), suf.end(), 0);
    sort();
    for (int k = 1; true; k *= 2) {
      for (int &i : suf) i = (i >= k) ? i - k : i - k + n + 1;
      sort();
      temp[suf[0]] = 0;
      for (int i = 1; i <= n; ++i) {
        int a = suf[i - 1], b = suf[i];
        temp[b] = temp[a] + (rank[a] != rank[b] or rank[a + k] !=
          rank[b + k]);
      }
      swap(rank, temp);
      if (rank[suf[n]] == n) break;
    }
    suf.erase(suf.begin()), rank.pop_back();
    for (int i = 0; i < n; ++i) rank[suf[i]] = i;
  }

  void longest_common_prefix() {
    lcp.assign(n, 0);
    for (int i = 0, h = 0; i < n; ++i) if (rank[i] > 0) {
      int j = suf[rank[i] - 1];
      while (i + h < n and j + h < n and s[i + h] == s[j + h]) h++;
      lcp[rank[i]] = h;
      if (h > 0) h--;
    }
  }

  array<int, 2> search(const string &t) {
    auto cmp = [&](int i) {return s.compare(i, t.size(), t);};
    int l = partition_point(suf.begin(), suf.end(), [&](int i)
      {return cmp(i) < 0;}) - suf.begin();
    int r = partition_point(suf.begin() + l, suf.end(), [&](int i)
      {return cmp(i) == 0;}) - suf.begin();
    return {l, r};
  }

  array<int, 2> kth_distinct(int k) {
    for (int i = 0; i < n; ++i) {
      int cnt = n - suf[i] - lcp[i];
      if (k < cnt) return {suf[i], lcp[i] + k + 1};
      k -= cnt;
    }
    return {-1, -1};
  }

  array<int, 2> kth(int k) {
    vector<int> pre(n + 1);
    for (int i = 0; i < n; ++i) pre[i + 1] = pre[i] + n - suf[i];
    if (k >= pre[n]) return {-1, -1};
    int l = 0, r = n, len = 0;
    while (true) {
      if (n - suf[l] == len) l++;
      char c = s[suf[l] + len];
      int m = partition_point(suf.begin() + l, suf.begin() + r,
        [&](int i) {return s[i + len] <= c;}) - suf.begin();
      int cnt = pre[m] - pre[l] - (m - l) * len;
      if (k >= cnt) {
        k -= cnt, l = m;
      } else {
        r = m, len++;
        if (k < r - l) return {suf[l], len};
        k -= r - l;
      }
    }
    return {-1, -1};
  }
};
```

tree-suffix-array

suffix-array/

```
// Tree Suffix Array: sorts the nodes by their string, the letters
  s[v] from v up to the root, with pattern search and lcp of any two
// Time: build O(N log N) | search O(M log N) |
  longest_common_prefix(a, b) O(log N) | less_equal O(log N) | Space:
  O(N log N)
// Note: root = 0 | suf = sorted nodes | rank[log][v] = number of
  distinct strings smaller than the string of v | search = [l, r)
struct tree_suffix_array {
  int n, log;
  string s;
  vector<int> suf, depth, lcp;
  vector<vector<int>> up, rank;

  tree_suffix_array(const string &s, const vector<vector<int>> &g):
    n(s.size()), s(s), suf(n + 1), depth(n + 1), lcp(n) {
    vector<int> p(n + 1, n), q = {0};
    depth[0] = 1;
    for (int i = 0; i < q.size(); ++i) {
      int a = q[i];
      for (int b : g[a]) if (b != p[a]) {
        p[b] = a, depth[b] = depth[a] + 1;
        q.push_back(b);
      }
    }
    vector<int> temp(n + 1), cnt(max<int>(256, n + 1));
    auto sort = [&](const vector<int> &key) {
      fill(cnt.begin(), cnt.end(), 0);
      for (int i = 0; i <= n; ++i) cnt[key[i]]++;
      for (int i = 1; i < cnt.size(); ++i) cnt[i] += cnt[i - 1];
      for (int i = n; i >= 0; --i) temp[--cnt[key[suf[i]]]] = suf[i];
      swap(suf, temp);
    };
    up = {p};
    rank = {vector<int>(n + 1)};
    for (int i = 0; i < n; ++i) rank[0][i] = s[i];
    iota(suf.begin(), suf.end(), 0);
    for (int k = 0; true; ++k) {
      vector<int> key(n + 1), nxt(n + 1), jump(n + 1);
      for (int i = 0; i <= n; ++i) key[i] = rank[k][up[k][i]];
      sort(key);
      sort(rank[k]);
      for (int i = 1; i <= n; ++i) {
        int a = suf[i - 1], b = suf[i];
        nxt[b] = nxt[a] + (rank[k][a] != rank[k][b] or key[a] !=
          key[b]);
      }
      rank.push_back(nxt);
      if (k > 0 and nxt[suf[n]] == rank[k][suf[n]]) break;
      for (int i = 0; i <= n; ++i) jump[i] = up[k][up[k][i]];
      up.push_back(jump);
    }
    log = up.size();
    suf.erase(suf.begin());
    for (int i = 1; i < n; ++i) lcp[i] = longest_common_prefix(suf[i
      - 1], suf[i]);
  }

  int longest_common_prefix(int a, int b) {
    if (rank[log][a] == rank[log][b]) return depth[a];
    int h = 0;
    for (int k = log - 1; k >= 0; --k) if (rank[k][a] == rank[k][b]) {
      h += 1 << k;
      a = up[k][a], b = up[k][b];
    }
    return h;
  }

  array<int, 2> search(const string &t) {
    auto cmp = [&](int i) {
      for (char c : t) {
        if (i == n or s[i] < c) return -1;
        if (s[i] > c) return 1;
        i = up[0][i];
      }
      return 0;
    };
    int l = partition_point(suf.begin(), suf.end(), [&](int i)
      {return cmp(i) < 0;}) - suf.begin();
    int r = partition_point(suf.begin() + l, suf.end(), [&](int i)
      {return cmp(i) == 0;}) - suf.begin();
    return {l, r};
  }

  int less_equal(const string &t) {
    auto cmp = [&](int i) {
      for (char c : t) {
        if (i == n or s[i] < c) return true;
        if (s[i] > c) return false;
        i = up[0][i];
      }
      return i == n;
    };
```

```
    return partition_point(suf.begin(), suf.end(), cmp) - suf.begin();
  }
};
```

suffix-automaton

```
// Suffix Automaton: smallest automaton that reads every substring of
  s
// Time: O(26 N) | Space: O(26 N)
// Note: root = 1 | each state groups substrings with the same end
  positions
struct suffix_automaton {
  struct node {
    int len = 0, pos = -1;
    int link = 0, cnt = 0;
    array<int, 26> next = {};
  };

  int last = 1;
  vector<node> tree = {node(), node()};

  void extend(char h) {
    int c = h - 'a', cur = tree.size(), i = last;
    tree.push_back({tree[last].len + 1, tree[last].len, 1, 1});
    last = cur;
    while (i != 0 and tree[i].next[c] == 0) {
      tree[i].next[c] = cur;
      i = tree[i].link;
    }
    if (i == 0) return;
    int j = tree[i].next[c];
    if (tree[i].len + 1 == tree[j].len) {
      tree[cur].link = j;
      return;
    }
    int clone = tree.size();
    tree.push_back(tree[j]);
    tree[clone].len = tree[i].len + 1;
    tree[clone].cnt = 0;
    while (i != 0 and tree[i].next[c] == j) {
      tree[i].next[c] = clone;
      i = tree[i].link;
    }
    tree[cur].link = tree[j].link = clone;
  }

  int find(const string &t) {
    int i = 1;
    for (char h : t) {
      int c = h - 'a';
      i = tree[i].next[c];
    }
    return i;
  }

  void count_occurrences() {
    vector<int> ord(tree.size() - 2); iota(ord.begin(), ord.end(), 2);
    sort(ord.rbegin(), ord.rend(), [&](int i, int j) {return
      tree[i].len < tree[j].len;});
    for (int i : ord) {
      tree[tree[i].link].cnt += tree[i].cnt;
    }
  }

  int distinct_substrings() {
    int dist = 0;
    for (int i = 2; i < tree.size(); ++i) {
      dist += tree[i].len - tree[tree[i].link].len;
    }
    return dist;
  }
};
```

trie

```
struct trie {
  struct node {
    int pref = 0, end = 0;
    array<int, 26> child = {};
  };

  vector<node> tree = {node()};

  int find(const string &s) {
    int a = 0;
    for (char h : s) {
      int c = h - 'a';
      a = tree[a].child[c];
      if (a == 0) return -1;
    }
    return a;
  }

  void update(const string &s, int d) {
    int a = 0;
    tree[a].pref += d;
    for (char h : s) {
      int c = h - 'a';
      if (tree[a].child[c] == 0) {
        tree[a].child[c] = tree.size();
        tree.emplace_back();
      }
      a = tree[a].child[c];
      tree[a].pref += d;
    }
    tree[a].end += d;
  }
};
```

z-array

```
vector<int> z_array(string &s) {
  int n = s.size();
  vector<int> z(n);
  for (int i = 1, l = 0, r = 0; i < n; ++i) {
    if (i < r) z[i] = min(r - i, z[i - l]);
    while (i + z[i] < n and s[z[i]] == s[i + z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  }
  return z;
}
```

structs

bit-2d

bit/

```
struct bit_2d {
  int n, m;
  vector<bit> tree;

  int lsb(int i) {
    return i & -i;
  }

  bit_2d(int n = 0, int m = 0): n(n), m(m), tree(n + 1, bit(m)) {}

  bit_2d(const vector<vector<int>> &a): bit_2d(a.size(), a[0].size())
    {
    for (int i = 1; i <= n; ++i) {
      bit row(a[i - 1]);
      for (int j = 1; j <= m; ++j) {
        tree[i].tree[j] += row.tree[j];
        if (i + lsb(i) <= n) tree[i + lsb(i)].tree[j] +=
          tree[i].tree[j];
      }
    }
  }

  void update(int r, int c, int x) {
    for (r += 1; r <= n; r += lsb(r)) tree[r].update(c, x);
  }

  int prefix(int r, int c) {
    int s = 0;
    for (r += 1; r > 0; r -= lsb(r)) s += tree[r].prefix(c);
    return s;
  }

  int query(int r1, int c1, int r2, int c2) {
    return prefix(r2, c2) - prefix(r1 - 1, c2) - prefix(r2, c1 - 1) +
      prefix(r1 - 1, c1 - 1);
  }
};
```

bit

bit/

```
struct bit {
  int n;
  vector<int> tree;

  int lsb(int i) {
    return i & -i;
  }

  bit(int n): n(n), tree(n + 1) {}

  void update(int i, int x) {
    for (i += 1; i <= n; i += lsb(i)) tree[i] += x;
  }

  int prefix(int i) {
    int s = 0;
    for (i += 1; i > 0; i -= lsb(i)) s += tree[i];
    return s;
  }

  int query(int i, int j) {
    return prefix(j) - prefix(i - 1);
  }

  int kth(int k) {
    int i = 0;
    for (int s = 1 << __lg(n); s > 0; s >>= 1) {
      if (i + s <= n and tree[i + s] <= k) i += s, k -= tree[i];
    }
    return i;
  }
};
```

range-bit

bit/

```
struct range_bit {
  bit m, s;

  range_bit(int n): m(n + 1), s(n + 1) {}

  void update(int i, int j, int x) {
    m.update(i, x);
    m.update(j + 1, -x);
    s.update(i, x * (i - 1));
    s.update(j + 1, -x * j);
  }

  int prefix(int i) {
    return i * m.prefix(i) - s.prefix(i);
  }

  int query(int i, int j) {
    return prefix(j) - prefix(i - 1);
  }
};
```

sparse-bit-2d

bit/

```
// Sparse BIT 2D: point update and rectangle sum over a fixed set of
  points
// Time: Build O(n log n) | Update/Query O(log^2 n) | Space: O(n log
  n)
struct sparse_bit_2d {
  int n;
  vector<int> rs;
  vector<vector<int>> cs, tree;

  static int lsb(int i) {
    return i & -i;
  }

  int id(const vector<int> &a, int x) {
    return upper_bound(a.begin(), a.end(), x) - a.begin();
  }

  sparse_bit_2d(vector<pair<int, int>> p) {
    for (auto [r, c] : p) rs.push_back(r);
    sort(rs.begin(), rs.end());
    rs.erase(unique(rs.begin(), rs.end()), rs.end());
    n = rs.size();
    cs.resize(n + 1), tree.resize(n + 1);
    sort(p.begin(), p.end(), [](auto &a, auto &b) { return a.second <
      b.second; });
    for (auto [r, c] : p) {
      for (int i = id(rs, r); i <= n; i += lsb(i)) {
        if (cs[i].empty() or cs[i].back() != c) cs[i].push_back(c);
      }
    }
    for (int i = 1; i <= n; ++i) tree[i].resize(cs[i].size() + 1);
  }

  void update(int r, int c, int v) {
    for (int i = id(rs, r); i <= n; i += lsb(i)) {
      for (int j = id(cs[i], c); j < tree[i].size(); j += lsb(j))
        tree[i][j] += v;
    }
  }

  int prefix(int r, int c) {
    int s = 0;
    for (int i = id(rs, r); i > 0; i -= lsb(i)) {
      for (int j = id(cs[i], c); j > 0; j -= lsb(j)) s += tree[i][j];
    }
    return s;
  }

  int query(int r1, int c1, int r2, int c2) {
    return prefix(r2, c2) - prefix(r1 - 1, c2) - prefix(r2, c1 - 1) +
      prefix(r1 - 1, c1 - 1);
  }
};
```

cover-tree

cover-tree/

```
// Cover Tree: calculates total length covered by at least one
  interval
// Time: update O(log N), query O(1) | Space: O(N)
struct cover_tree {
```

```
  int n;
  vector<int> all, cnt, curr;

  void recalc(int i) {
    if (cnt[i] > 0) curr[i] = all[i];
    else if (i < n) curr[i] = curr[i << 1] + curr[i << 1 | 1];
    else curr[i] = 0;
  }

  cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2 *
    n), curr(2 * n) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j, int x) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] += x, recalc(i++);
      if (j & 1) cnt[--j] += x, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1];
  }
};

// XOR Cover Tree: calculates total length covered by an odd number
  of intervals
// Time: update O(log N), query O(1) | Space: O(N)
struct xor_cover_tree {
  int n;
  vector<int> all, cnt, curr;

  void recalc(int i) {
    int child = 0;
    if (i < n) child = curr[i << 1] + curr[i << 1 | 1];
    if (cnt[i] & 1) curr[i] = all[i] - child;
    else curr[i] = child;
  }

  xor_cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2
    * n), curr(2 * n) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] ^= 1, recalc(i++);
      if (j & 1) cnt[--j] ^= 1, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1];
  }
};
```

k-cover-tree

cover-tree/

```
// Iterative K-Cover Segment Tree: calculates total length covered by
  at least K intervals
// Time: update O(K log N), query O(1) | Space: O(N * K)
struct k_cover_tree {
  int n, k;
  vector<int> all, cnt;
  vector<vector<int>> curr;

  void recalc(int i) {
    vector<int> base(k + 1);
    base[0] = all[i];
    if (i < n) for (int h = 1; h <= k; ++h) base[h] = curr[i << 1][h]
      + curr[i << 1 | 1][h];
    for (int h = 1; h <= k; ++h) curr[i][h] = base[max<int>(0, h -
      cnt[i])];
  }

  k_cover_tree(int k, vector<int> &x): n(x.size() - 1), k(k), all(2 *
    n), cnt(2 * n), curr(2 * n, vector<int>(k + 1)) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1
      | 1];
  }

  void update(int i, int j, int x) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >>= 1, j >>= 1) {
      if (i & 1) cnt[i] += x, recalc(i++);
      if (j & 1) cnt[--j] += x, recalc(j);
    }
    i = p + n;
    while (i >>= 1) recalc(i);
    j = q + n - 1;
    while (j >>= 1) recalc(j);
  }

  int query() {
    return curr[1][k];
  }
};
```

coprime-set

custom-sets/

```
// Coprime Set: counts elements coprime to a given number using
  Inclusion-Exclusion
// Time: insert/remove/query O(2^K * (K + log D)) | Space: O(N * 2^K)
// Note: P must be a vector of DISTINCT prime factors | K = size of
  P, D = unique divisors in map
struct coprime_set {
  map<int, int> n;

  vector<pair<int, int>> divisors(vector<int> &p) {
    int k = p.size();
    vector<pair<int, int>> a; a.reserve(1 << k);
    for (int m = 0; m < (1 << k); ++m) {
      int d = 1, mu = 1;
      for (int i = 0; i < k; ++i) if (m & (1 << i)) {
        d *= p[i], mu *= -1;
      }
      a.push_back({d, mu});
    }
    return a;
  }

  void insert(vector<int> &p) {
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      n[d]++;
    }
  }

  void remove(vector<int> &p) {
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      n[d]--;
    }
  }

  int query(vector<int> &p) {
    int t = 0;
    vector<pair<int, int>> a = divisors(p);
    for (auto &[d, mu] : a) {
      t += n[d] * mu;
    }
    return t;
  }
};
```

interval-set

custom-sets/

```
// Interval Set: maintains disjoint intervals with dynamic active
  elements count
// Time: insert/remove O(log N) amortized | contains O(log N) | total
  O(1) | Space: O(N)
// Note: discrete=true for [l, r] (r-l+1) | discrete=false for
  continuous [l, r) (r-l)
struct interval_set {
  int active = 0, off;
  set<pair<int, int>> s;

  interval_set(bool discrete): off(discrete) {}

  void insert(int l, int r) {
    assert(l < r + off);
    auto itr = s.lower_bound({l, -inf});
    if (itr != s.begin() and prev(itr)->second >= l - off) itr--;
    int nl = l;
    int nr = r;
    while (itr != s.end() and itr->first <= r + off) {
      nl = min(nl, itr->first);
      nr = max(nr, itr->second);
      active -= itr->second - itr->first + off;
      itr = s.erase(itr);
    }
    active += nr - nl + off;
    s.insert({nl, nr});
  }

  void remove(int l, int r) {
    assert(l < r + off);
    auto itr = s.lower_bound({l, -inf});
    if (itr != s.begin() and prev(itr)->second >= l) itr--;
    while (itr != s.end() and itr->first <= r) {
      int cl = itr->first;
      int cr = itr->second;
      active -= cr - cl + off;
      itr = s.erase(itr);
      if (cl < l) {
        s.insert({cl, l - off});
        active += l - cl;
      }
      if (cr > r) {
        s.insert({r + off, cr});
        active += cr - r;
      }
    }
  }

  bool contains(int l, int r) {
    assert(l < r + off);
    auto itr = s.upper_bound({l, inf});
    if (itr == s.begin()) return false;
    itr--;
    return itr->first <= l and r <= itr->second;
  }

  int total() {
    return active;
  }
};

// Color Set: disjoint intervals [l, r], each with a color, painting
  a range and returning what was there before, with lazy moves of all
  positions
// Time: paint and erase O(log N) amortized | contains and color
  O(log N) | shift and flip O(1) | Space: O(N)
// Note: real position = sign * stored + offset | shift(k) = x -> x +
  k, flip(c) = x -> c - x (flip(l + r) mirrors [l, r])
struct color_set {
  struct node {
    int l, r, color;
  };

  map<int, node> s;
  int sign = 1, offset = 0;

  int stored(int x) {
    return sign * (x - offset);
  }

  node real(node a) {
    int l = sign * a.l + offset, r = sign * a.r + offset;
    a.l = min(l, r), a.r = max(l, r);
    return a;
  }

  void split(int x) {
    auto it = s.lower_bound(x);
    if (it == s.begin()) return;
    node &a = (--it)->second;
    if (x <= a.r) {
      node b = a;
      b.l = x, a.r = x - 1;
      s[x] = b;
    }
  }

  vector<node> erase(int l, int r) {
    if (l > r) return {};
    int a = min(stored(l), stored(r)), b = max(stored(l), stored(r));
    split(a), split(b + 1);
    vector<node> removed;
    auto first = s.lower_bound(a), last = s.lower_bound(b + 1);
    for (auto it = first; it != last; ++it)
      removed.push_back(real(it->second));
    s.erase(first, last);
    if (sign == -1) reverse(removed.begin(), removed.end());
    return removed;
  }

  vector<node> paint(int l, int r, int c) {
    if (l > r) return {};
    vector<node> removed = erase(l, r);
```

```
    int a = min(stored(l), stored(r)), b = max(stored(l), stored(r));
    s[a] = {a, b, c};
    return removed;
  }

  bool contains(int x) {
    auto it = s.upper_bound(stored(x));
    return it != s.begin() and prev(it)->second.r >= stored(x);
  }

  int color(int x) {
    return prev(s.upper_bound(stored(x)))->second.color;
  }

  void shift(int k) {
    offset += k;
  }

  void flip(int c) {
    sign = -sign, offset = c - offset;
  }
};
```

range-set

custom-sets/

```
// Range Set: bitset optimized for range updates and queries
// Time: O(N/64) per operation | Space: O(N/64)
// Note: uint = uint64_t (unsigned long long)
struct range_set {
  int n;
  vector<uint> data;

  range_set(int n): n(n), data((n + 63) / 64) {}

  uint from(int i) {
    return ~0ULL << (i & 63);
  }

  uint until(int i) {
    return ~0ULL >> (63 - (i & 63));
  }

  void apply(int k, bool on, uint mask) {
    if (on) data[k] |= mask;
    else data[k] &= ~mask;
  }

  void update(int l, int r, bool on) {
    int i = l >> 6, j = r >> 6;
    if (i == j) {
      apply(i, on, from(l) & until(r));
    } else {
      apply(i, on, from(l));
      for (int k = i + 1; k < j; ++k) apply(k, on, ~0ULL);
      apply(j, on, until(r));
    }
  }

  int count(int l, int r) {
    int c = 0;
    int i = l >> 6, j = r >> 6;
    if (i == j) {
      c = __builtin_popcountll(data[i] & (from(l) & until(r)));
    } else {
      c += __builtin_popcountll(data[i] & from(l));
      for (int k = i + 1; k < j; ++k) c +=
        __builtin_popcountll(data[k]);
      c += __builtin_popcountll(data[j] & until(r));
    }
    return c;
  }

  bool empty() {
    for (uint mask : data) if (mask) return false;
    return true;
  }

  int prev(int i) {
    for (int k = i >> 6; k >= 0; --k, i = 63) {
      uint mask = data[k] & until(i);
      if (mask) return (k << 6) + 63 - __builtin_clzll(mask);
    }
    return n;
  }

  int next(int i) {
    for (int k = i >> 6; k < data.size(); ++k, i = 0) {
      uint mask = data[k] & from(i);
      if (mask) return (k << 6) + __builtin_ctzll(mask);
    }
    return n;
  }

  int segments() {
    int s = 0;
    for (int k = 0; k < data.size(); ++k) {
      s += __builtin_popcountll((data[k] >> 1) & ~data[k]);
      if ((data[k] & 1) and (k == 0 or (data[k - 1] >> 63) == 0)) s
        += 1;
    }
    return s;
  }
};
```

venice-set

custom-sets/

```
// Venice Set: multiset with global value updates
// Time: O(log N) | Space: O(N)
struct venice_set {
  int k = 0;
  multiset<int> s;

  void insert(int x) {
    s.insert(x - k);
  }

  void remove(int x) {
    if (auto i = s.find(x - k); i != s.end()) s.erase(i);
  }

  void update(int x) {
    k += x;
  }

  int min() {
    return *s.begin() + k;
  }
};
```

dsu-2d

dsu/

```
struct dsu_2d {
  int n, m;
  dsu d;

  dsu_2d(int n, int m): n(n), m(m), d(n * m) {}

  int id(int r, int c) {
    return r * m + c;
  }

  int find(int r, int c) {
    return d.find(id(r, c));
  }

  void unite(int r1, int c1, int r2, int c2) {
    d.unite(id(r1, c1), id(r2, c2));
  }
};
```

dsu

dsu/

```
struct dsu {
  vector<int> p, s;

  dsu(int n): p(n), s(n, 1) {
    iota(p.begin(), p.end(), 0);
  }

  int find(int a) {
    if (p[a] == a) return a;
    return p[a] = find(p[a]);
  }

  void unite(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b) return;
    if (s[a] < s[b]) swap(a, b);
    p[b] = a;
    s[a] += s[b];
  }
};
```

dynamic-connectivity

dsu/

```
// Dynamic Connectivity: offline number of components at each time,
  with edges alive in time intervals
// Time: O(q + k log q log n) | Space: O(n + q + k log q), with k
  calls to insert
// Note: requires rollback dsu; insert(a, b, l, r) means edge (a, b)
  exists at times l..r of [0, q - 1]
struct dynamic_connectivity {
  int n, q;
  vector<vector<pair<int, int>>> tree;

  dynamic_connectivity(int n, int q): n(n), q(q), tree(4 * q) {}

  void insert(int a, int b, int ql, int qr) {
    auto rec = [&](auto &self, int p, int l, int r) -> void {
      if (qr < l or r < ql) return;
      if (ql <= l and r <= qr) return tree[p].push_back({a, b});
      int m = l + (r - l) / 2;
      self(self, 2 * p, l, m);
      self(self, 2 * p + 1, m + 1, r);
    };
    rec(rec, 1, 0, q - 1);
  }

  vector<int> compute() {
    rollback_dsu dsu(n);
    vector<int> ans(q);
    auto rec = [&](auto &self, int p, int l, int r) -> void {
      int t = dsu.state();
      for (auto [a, b] : tree[p]) dsu.unite(a, b);
      if (l == r) {
        ans[l] = dsu.c;
      } else {
        int m = l + (r - l) / 2;
        self(self, 2 * p, l, m);
        self(self, 2 * p + 1, m + 1, r);
      }
      while (dsu.state() > t) dsu.rollback();
    };
    rec(rec, 1, 0, q - 1);
    return ans;
  }
};
```

rollback-dsu

dsu/

```
struct rollback_dsu {
  int c;
  vector<int> p, s, h;

  rollback_dsu(int n): c(n), p(n), s(n, 1) {
    iota(p.begin(), p.end(), 0);
  }

  int find(int a) {
    while (p[a] != a) a = p[a];
    return a;
  }

  bool unite(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return h.push_back(-1), false;
    if (s[a] < s[b]) swap(a, b);
    p[b] = a, s[a] += s[b], c--;
    h.push_back(b);
    return true;
  }

  void rollback() {
    int b = h.back();
    h.pop_back();
    if (b == -1) return;
    int a = p[b];
    p[b] = b, s[a] -= s[b], c++;
  }

  int state() {
    return h.size();
  }
};
```

dynamic-merge-sort-tree

merge-sort-tree/

```
// Dynamic Merge Sort Tree: range queries with point updates
// Time: build O(N log^2 N) | Update/Query: O(log^2 N) | Space: O(N
  log N)
// Note: uses PBDS ordered_set | pair<int, int> handles duplicate
  values
struct dynamic_merge_sort_tree {
  int n;
  vector<int> a;
  vector<ordered_set<pair<int, int>>> tree;

  dynamic_merge_sort_tree(vector<int> &a): n(a.size()), a(a), tree(2
    * n) {
    for (int i = 0; i < n; ++i) {
      for (int j = i + n; j > 0; j >>= 1) {
        tree[j].insert({a[i], i});
      }
    }
  }

  void update(int i, int x) {
    for (int j = i + n; j > 0; j >>= 1) {
      tree[j].erase({a[i], i});
      tree[j].insert({x, i});
```

```
    }
    a[i] = x;
  }

  int less_or_equal(int i, int j, int x) {
    int c = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) c += tree[i++].order_of_key({x + 1, 0});
      if (j & 1) c += tree[--j].order_of_key({x + 1, 0});
    }
    return c;
  }
};
```

merge-sort-tree

merge-sort-tree/

```
struct merge_sort_tree {
  int n;
  vector<vector<int>> tree;

  merge_sort_tree(vector<int> &a): n(a.size()), tree(2 * n) {
    for (int i = 0; i < n; ++i) tree[i + n] = {a[i]};
    for (int i = n - 1; i > 0; --i) merge(tree[i << 1].begin(),
      tree[i << 1].end(), tree[(i << 1) | 1].begin(), tree[(i << 1) |
      1].end(), back_inserter(tree[i]));
  }

  int less_or_equal(int i, int j, int x) {
    int c = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) c += upper_bound(tree[i].begin(), tree[i].end(), x)
        - tree[i].begin(), i++;
      if (j & 1) --j, c += upper_bound(tree[j].begin(),
        tree[j].end(), x) - tree[j].begin();
    }
    return c;
  }
};
```

ordered-set

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
  tree_order_statistics_node_update>;
```

iterative-segment-tree-2d

segment-tree/

```
struct segment_tree_2d {
  int n, m;
  vector<segment_tree> tree;

  segment_tree_2d(int r, int c): n(__bit_ceil(r)), m(__bit_ceil(c)),
    tree(2 * n, segment_tree(m)) {}

  segment_tree_2d(vector<vector<int>> &a): segment_tree_2d(a.size(),
    a[0].size()) {
    for (int i = 0; i < a.size(); ++i) tree[n + i] =
      segment_tree(a[i]);
    for (int i = n - 1; i > 0; --i) {
      for (int j = 1; j < 2 * m; ++j) {
        tree[i].tree[j] = segment_tree::merge(tree[i << 1].tree[j],
          tree[(i << 1) | 1].tree[j]);
      }
    }
  }

  void update(int i, int j, int x) {
    tree[i += n].update(j, x);
    while (i >>= 1) tree[i].update(j, segment_tree::merge(tree[i <<
      1].tree[j + m], tree[(i << 1) | 1].tree[j + m]));
  }

  int query(int i1, int j1, int i2, int j2) {
    int l = 0, r = 0;
    for (i1 += n, i2 += n + 1; i1 < i2; i1 >>= 1, i2 >>= 1) {
      if (i1 & 1) l = segment_tree::merge(l, tree[i1++].query(j1,
        j2));
      if (i2 & 1) r = segment_tree::merge(tree[--i2].query(j1, j2),
        r);
    }
    return segment_tree::merge(l, r);
  }
};
```

iterative-segment-tree

segment-tree/

```
struct segment_tree {
  int n;
  vector<int> tree;

  static int merge(int x, int y) {
    return x + y;
  }

  segment_tree(int s): n(__bit_ceil(s)), tree(2 * n) {}

  segment_tree(vector<int> &a): segment_tree(a.size()) {
    for (int i = 0; i < a.size(); ++i) tree[n + i] = a[i];
    for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1],
      tree[(i << 1) | 1]);
  }

  void update(int i, int x) {
    i += n;
    tree[i] = x;
    while (i >>= 1) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
  }

  int query(int i, int j) {
    int l = 0, r = 0;
    for (i += n, j += n + 1; i < j; i >>= 1, j >>= 1) {
      if (i & 1) l = merge(l, tree[i++]);
      if (j & 1) r = merge(tree[--j], r);
    }
    return merge(l, r);
  }
};
```

segment-tree-beats

segment-tree/

Segment Tree Beats

Ideia. É uma seg com lazy em que o update nem sempre para no nó coberto pelo intervalo. Cada nó guarda informação extra que diz se dá para aplicar o update no nó inteiro de uma vez; se não dá, ele desce mais. Uma análise amortizada garante que essas descidas extras são raras. A query não muda.

```
if (fora do intervalo or break) return;
if (dentro do intervalo and tag) return aplica_tag(p);
push, desce nos dois filhos, pull;
```

chmin: $a_i = \min(a_i, x)$

- **Nó:** soma, max_1 (maior valor), max_2 (segundo maior, estrito) e c (quantas vezes max_1 aparece).
- **Merge:** max_1 é o maior dos dois; max_2 é o maior valor diferente de max_1 entre os quatro; c soma os c dos filhos que têm o max_1.
- **Break:** $x \geq max_1$, nada muda.
- **Tag:** $max_2 < x < max_1$, só os máximos mudam: soma $-= (max_1 - x) \cdot c$ e $max_1 = x$.
- **Push:** o max_1 do pai já é o lazy. Em cada filho com max_1 maior que o do pai, aplique a tag com $x = max_1$ do pai.
- **Custo:** $O((n+q)\log n)$ amortizado.

chmax e os dois juntos

O chmax é simétrico, com min_1, min_2 e a contagem do mínimo. Com os dois na mesma seg, um nó com só um ou dois valores distintos tem máximo e mínimo se encostando: ao trocar max_1 por x, troque também min_1 (se era igual a max_1) ou min_2 (se era igual a max_1), e vice-versa.

Com soma em intervalo

Um lazy de soma comum. Somar v num nó soma $v \cdot len$ na soma e v em $max_1, max_2, min_1, min_2$, sem mexer nos que valem $\pm\infty$. No push, aplique primeiro a soma e depois o chmin e o chmax com o max_1 e o min_1 do pai. O custo passa a ser $O(\log^2 n)$ amortizado por operação.

Updates que só valem com um valor só

Mod, divisão, raiz e afins não têm uma tag para valores diferentes. Guarde máximo e mínimo:

- **Break:** a operação não muda nada no nó (mod x com $max < x$; divisão com $x = 1$ ou $max = 0$; raiz com $max \leq 1$).
- **Tag:** $max = min$, todos os valores são iguais e mudam igual: vira somar $f(max) - max$ no nó.
- **Custo:** cada valor cai pela metade a cada mod ou divisão efetiva (e a raiz chega a 1 em $O(\log\log C)$ passos), então são $O(\log C)$ descidas por elemento. Com soma em intervalo, o custo continua amortizado.
- **AND e OR:** guarde o AND e o OR do nó. Break se nenhum bit muda; tag se todos os valores concordam nos bits que mudam (AND e OR do nó iguais nesses bits), porque aí todos mudam pelo mesmo valor.

O que mudar numa seg normal

1. Os campos novos no nó e o merge deles.
2. No update, a condição de break junto do teste de fora do intervalo, e a de tag junto do teste de cobertura.
3. No push, além do lazy normal, aplicar nos filhos o corte que o pai já sofreu.

segment-tree

segment-tree/

```
struct segment_tree {
  struct tag {
    int x = 0;

    bool operator==(const tag &o) const {
      return x == o.x;
    }
  };

  struct node {
    int sum = 0;
    tag lazy;

    node operator+(const node &o) const {
      node c;
      c.sum = sum + o.sum;
      return c;
    }

    void apply(const tag &t, int len) {
      sum += t.x * len;
      lazy.x += t.x;
    }

    bool good(int x) const {
      return sum >= x;
    }
  };

  int n;
  vector<node> tree;

  segment_tree(int n): n(n), tree(4 * n) {}

  void push(int p, int l, int r) {
    if (tree[p].lazy == tag()) return;
    int m = l + (r - l) / 2;
    tree[2 * p].apply(tree[p].lazy, m - l + 1);
    tree[2 * p + 1].apply(tree[p].lazy, r - m);
    tree[p].lazy = tag();
  }

  void pull(int p) {
    tree[p] = tree[2 * p] + tree[2 * p + 1];
  }

  void build(const vector<node> &a) {
    auto rec = [&](auto &self, int p, int l, int r) -> void {
      if (l == r) {
        tree[p] = a[l];
        return;
      }
      int m = l + (r - l) / 2;
      self(self, 2 * p, l, m);
      self(self, 2 * p + 1, m + 1, r);
      pull(p);
    };
    rec(rec, 1, 0, n - 1);
  }

  void set(int i, const node &x) {
    auto rec = [&](auto &self, int p, int l, int r) -> void {
      if (l == r) {
        tree[p] = x;
        return;
      }
      push(p, l, r);
      int m = l + (r - l) / 2;
      if (i <= m) self(self, 2 * p, l, m);
      else self(self, 2 * p + 1, m + 1, r);
      pull(p);
    };
    rec(rec, 1, 0, n - 1);
  }

  void update(int ql, int qr, const tag &t) {
    auto rec = [&](auto &self, int p, int l, int r) -> void {
      if (qr < l or r < ql) return;
      if (ql <= l and r <= qr) return tree[p].apply(t, r - l + 1);
      push(p, l, r);
      int m = l + (r - l) / 2;
      self(self, 2 * p, l, m);
      self(self, 2 * p + 1, m + 1, r);
      pull(p);
    };
    rec(rec, 1, 0, n - 1);
  }

  node query(int ql, int qr) {
    auto rec = [&](auto &self, int p, int l, int r) -> node {
```

```
      if (qr < l or r < ql) return node();
      if (ql <= l and r <= qr) return tree[p];
      push(p, l, r);
      int m = l + (r - l) / 2;
      return self(self, 2 * p, l, m) + self(self, 2 * p + 1, m + 1,
        r);
    };
    return rec(rec, 1, 0, n - 1);
  }

  int left(int qr, int x) {
    node acc;
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      if (qr < l) return l - 1;
      node c = tree[p] + acc;
      if (r <= qr and not c.good(x)) return acc = c, l - 1;
      if (l == r) return l;
      push(p, l, r);
      int m = l + (r - l) / 2, i = self(self, 2 * p + 1, m + 1, r);
      return i > m ? i : self(self, 2 * p, l, m);
    };
    return rec(rec, 1, 0, n - 1);
  }

  int right(int ql, int x) {
    node acc;
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      if (r < ql) return r + 1;
      node c = acc + tree[p];
      if (ql <= l and not c.good(x)) return acc = c, r + 1;
      if (l == r) return l;
      push(p, l, r);
      int m = l + (r - l) / 2, i = self(self, 2 * p, l, m);
      return i <= m ? i : self(self, 2 * p + 1, m + 1, r);
    };
    return rec(rec, 1, 0, n - 1);
  }
};
```

sparse-segment-tree

segment-tree/

```
struct sparse_segment_tree {
  struct tag {
    int x = 0;

    bool operator==(const tag &o) const {
      return x == o.x;
    }
  };

  struct node {
    int sum = 0;
    tag lazy;
    int l = 0, r = 0;

    node operator+(const node &o) const {
      node c;
      c.sum = sum + o.sum;
      return c;
    }

    void apply(const tag &t, int len) {
      sum += t.x * len;
      lazy.x += t.x;
    }

    bool good(int x) const {
      return sum >= x;
    }
  };

  int low, high, root = 0;
  bool keep;
  vector<node> tree = {node()};

  sparse_segment_tree(int low, int high, bool keep): low(low),
    high(high), keep(keep) {}

  int touch(int p) {
    if (p and not keep) return p;
    tree.push_back(tree[p]);
    return tree.size() - 1;
  }

  void push(int p, int l, int r) {
    if (tree[p].lazy == tree[0].lazy) return;
    int m = l + (r - l) / 2;
    tree[p].l = touch(tree[p].l), tree[p].r = touch(tree[p].r);
    tag t = tree[p].lazy;
    tree[tree[p].l].apply(t, m - l + 1);
    tree[tree[p].r].apply(t, r - m);
    tree[p].lazy = tree[0].lazy;
  }

  void pull(int p) {
    node c = tree[tree[p].l] + tree[tree[p].r];
    c.l = tree[p].l, c.r = tree[p].r;
    tree[p] = c;
  }

  int set(int i, const node &x, int v = -1) {
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      p = touch(p);
      if (l == r) return tree[p] = x, p;
      push(p, l, r);
      int m = l + (r - l) / 2;
      if (i <= m) tree[p].l = self(self, tree[p].l, l, m);
      else tree[p].r = self(self, tree[p].r, m + 1, r);
      pull(p);
      return p;
    };
    return root = rec(rec, v < 0 ? root : v, low, high);
  }

  int update(int ql, int qr, const tag &t, int v = -1) {
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      if (qr < l or r < ql) return p;
      p = touch(p);
      if (ql <= l and r <= qr) return tree[p].apply(t, r - l + 1), p;
      push(p, l, r);
      int m = l + (r - l) / 2;
      tree[p].l = self(self, tree[p].l, l, m);
      tree[p].r = self(self, tree[p].r, m + 1, r);
      pull(p);
      return p;
    };
    return root = rec(rec, v < 0 ? root : v, low, high);
  }

  node query(int ql, int qr, int v = -1) {
    auto rec = [&](auto &self, int p, int l, int r) -> node {
      if (qr < l or r < ql) return node();
      if (not p or (ql <= l and r <= qr)) return tree[p];
      push(p, l, r);
      int m = l + (r - l) / 2;
      return self(self, tree[p].l, l, m) + self(self, tree[p].r, m +
        1, r);
    };
    return rec(rec, v < 0 ? root : v, low, high);
  }

  int left(int qr, int x, int v = -1) {
    node acc;
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      if (qr < l) return l - 1;
      node c = tree[p] + acc;
      if (r <= qr and not c.good(x)) return acc = c, l - 1;
      if (l == r) return l;
      push(p, l, r);
      int m = l + (r - l) / 2, i = self(self, tree[p].r, m + 1, r);
      return i > m ? i : self(self, tree[p].l, l, m);
    };
    return rec(rec, v < 0 ? root : v, low, high);
  }

  int right(int ql, int x, int v = -1) {
    node acc;
    auto rec = [&](auto &self, int p, int l, int r) -> int {
      if (r < ql) return r + 1;
      node c = acc + tree[p];
      if (ql <= l and not c.good(x)) return acc = c, r + 1;
      if (l == r) return l;
      push(p, l, r);
      int m = l + (r - l) / 2, i = self(self, tree[p].l, l, m);
      return i <= m ? i : self(self, tree[p].r, m + 1, r);
    };
    return rec(rec, v < 0 ? root : v, low, high);
  }
};
```

sparse-table-2d

sparse-table/

```
// Sparse Table 2D: rectangle queries on a static matrix, for an
  idempotent merge like min, max, gcd, and, or
// Time: O(N M log N log M) build, O(1) query | Space: O(N M log N
  log M)
// Note: requires sparse_table, whose merge is used | query(r1, c1,
  r2, c2) covers rows [r1, r2] and columns [c1, c2]
struct sparse_table_2d {
  vector<vector<sparse_table>> table;

  sparse_table_2d(vector<vector<int>> &a): table(1) {
    for (auto &row : a) table[0].emplace_back(row);
    for (int k = 1; (1 << k) <= a.size(); ++k) {
      table.emplace_back();
      for (int i = 0; i + (1 << k) <= a.size(); ++i) {
        vector<int> row = table[k - 1][i].table[0];
        auto &other = table[k - 1][i + (1 << (k - 1))].table[0];
        for (int j = 0; j < row.size(); ++j) row[j] =
          sparse_table::merge(row[j], other[j]);
        table[k].emplace_back(row);
      }
    }
  }

  int query(int r1, int c1, int r2, int c2) {
    int k = __lg(r2 - r1 + 1);
    return sparse_table::merge(table[k][r1].query(c1, c2),
      table[k][r2 - (1 << k) + 1].query(c1, c2));
  }
};
```

sparse-table

sparse-table/

```
// Sparse Table: range queries on a static array, for an idempotent
  merge like min, max, gcd, and, or
// Time: O(N log N) build, O(1) query | Space: O(N log N)
// Note: query(l, r) covers [l, r] | merge(x, x) must be x, since the
  two halves of a query overlap
struct sparse_table {
  vector<vector<int>> table;

  static int merge(int x, int y) {
    return gcd(x, y);
  }

  sparse_table(vector<int> &a): table(1, a) {
    for (int k = 1; (1 << k) <= a.size(); ++k) {
      table.emplace_back(a.size() - (1 << k) + 1);
      for (int i = 0; i < table[k].size(); ++i) {
        table[k][i] = merge(table[k - 1][i], table[k - 1][i + (1 <<
          (k - 1))]);
      }
    }
  }

  int query(int l, int r) {
    int k = __lg(r - l + 1);
    return merge(table[k][l], table[k][r - (1 << k) + 1]);
  }
};
```

treap

```
// Treap: dynamic array with insert, erase, set, move, reverse, range
  update and range query
// Time: O(log N) expected per operation | Space: O(N) | needs rng
  (mt19937) | edit node, pull and apply
// Note: 0-based positions | less(x) = count < x, only if the array
  is kept sorted with insert(less(x), x)
struct treap {
  struct node {
    int val = 0, pri = rng();
    int sum = 0, lazy = 0;
    bool rev = false;
    int size = 0, l = 0, r = 0;
  };

  void pull(int n) {
    node &a = tree[n], &l = tree[a.l], &r = tree[a.r];
    a.size = l.size + 1 + r.size;
    a.sum = l.sum + a.val + r.sum;
  }

  void apply(int n, int x) {
    if (n == 0) return;
    tree[n].val += x;
    tree[n].sum += x * tree[n].size;
    tree[n].lazy += x;
  }

  int root = 0;
  vector<node> tree = {node()};

  treap(const vector<int> &a = {}) {
    for (int x : a) root = merge(root, make(x));
  }

  int make(int x) {
    tree.push_back({x});
    pull(tree.size() - 1);
    return tree.size() - 1;
  }

  void flip(int n) {
    if (n == 0) return;
    swap(tree[n].l, tree[n].r);
```

```
  tree[n].rev ^= 1;
}

void push(int n) {
  if (tree[n].rev) flip(tree[n].l), flip(tree[n].r), tree[n].rev =
    false;
  if (tree[n].lazy != tree[0].lazy) {
    apply(tree[n].l, tree[n].lazy), apply(tree[n].r, tree[n].lazy);
    tree[n].lazy = tree[0].lazy;
  }
}

array<int, 2> split(int n, int k) {
  if (n == 0) return {0, 0};
  push(n);
  int s = tree[tree[n].l].size;
  if (k <= s) {
    auto [a, b] = split(tree[n].l, k);
    tree[n].l = b;
    pull(n);
    return {a, n};
  }
  auto [a, b] = split(tree[n].r, k - s - 1);
  tree[n].r = a;
  pull(n);
  return {n, b};
}

int merge(int a, int b) {
  if (a == 0 or b == 0) return a | b;
  if (tree[a].pri > tree[b].pri) {
    push(a);
    tree[a].r = merge(tree[a].r, b);
    pull(a);
    return a;
  }
  push(b);
  tree[b].l = merge(a, tree[b].l);
  pull(b);
  return b;
}

array<int, 3> split3(int l, int r) {
  auto [a, b] = split(root, l);
  auto [m, c] = split(b, r - l + 1);
  return {a, m, c};
}

void merge3(int a, int m, int c) {
  root = merge(merge(a, m), c);
}

void insert(int i, int x) {
  auto [a, b] = split(root, i);
  root = merge(merge(a, make(x)), b);
}

void erase(int l, int r) {
  auto [a, m, c] = split3(l, r);
  root = merge(a, c);
}

void set(int i, int x) {
  auto [a, m, c] = split3(i, i);
  tree[m].val = x;
  pull(m);
  merge3(a, m, c);
}

void move(int l, int r, int k) {
  auto [a, m, c] = split3(l, r);
  auto [d, e] = split(merge(a, c), k);
  root = merge(merge(d, m), e);
}

void reverse(int l, int r) {
  auto [a, m, c] = split3(l, r);
  flip(m);
  merge3(a, m, c);
}

void update(int l, int r, int x) {
  auto [a, m, c] = split3(l, r);
  apply(m, x);
  merge3(a, m, c);
}

node query(int l, int r) {
  auto [a, m, c] = split3(l, r);
  node q = tree[m];
  merge3(a, m, c);
    return q;
  }

  int less(int x) {
    int i = 0, n = root;
    while (n != 0) {
      push(n);
      if (tree[n].val < x) i += tree[tree[n].l].size + 1, n =
        tree[n].r;
      else n = tree[n].l;
    }
    return i;
  }
};
```

wavelet-matrix

```
// Wavelet Matrix: range quantile, range frequency, k-th smallest
// Time: build O(N log V) | Queries O(log V) | Space: O(N log V)
// Note: kth is 0-indexed
struct wavelet_matrix {
  int n, log = 1;
  vector<int> vals, zeros;
  vector<vector<int>> pref;

  wavelet_matrix(vector<int> a): n(a.size()), vals(a) {
    sort(vals.begin(), vals.end());
    vals.erase(unique(vals.begin(), vals.end()), vals.end());
    for (int &x : a) x = lower_bound(vals.begin(), vals.end(), x) -
      vals.begin();
    while ((1 << log) < vals.size()) log++;
    zeros.resize(log);
    pref.assign(log, vector<int>(n + 1));
    for (int b = log - 1; b >= 0; --b) {
      for (int i = 0; i < n; ++i) pref[b][i + 1] = pref[b][i] +
        !(a[i] >> b & 1);
      zeros[b] = stable_partition(a.begin(), a.end(), [&](int x) {
        return !(x >> b & 1); }) - a.begin();
    }
  }

  int kth(int l, int r, int k) {
    int x = 0;
    r++;
    for (int b = log - 1; b >= 0; --b) {
      int zl = pref[b][l], zr = pref[b][r];
      if (k < zr - zl) l = zl, r = zr;
      else x |= 1 << b, k -= zr - zl, l += zeros[b] - zl, r +=
        zeros[b] - zr;
    }
    return vals[x];
  }

  int less_equal(int l, int r, int x) {
    int t = upper_bound(vals.begin(), vals.end(), x) - vals.begin(),
      c = 0;
    if (t == (1 << log)) return r - l + 1;
    r++;
    for (int b = log - 1; b >= 0; --b) {
      int zl = pref[b][l], zr = pref[b][r];
      if (t >> b & 1) c += zr - zl, l += zeros[b] - zl, r += zeros[b]
        - zr;
      else l = zl, r = zr;
    }
    return c;
  }
};
```